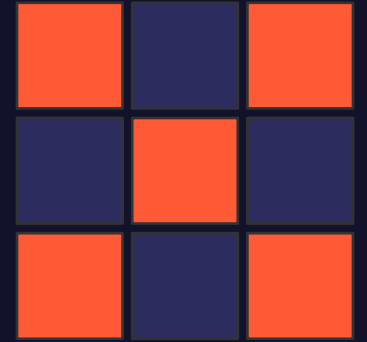


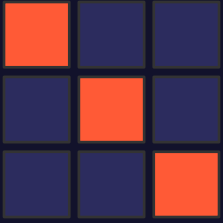
# Attention and Vision Transformers

Building the attention mechanism from first principles, then assembling it into ViT — with the emphasis on intuition rather than derivation.

---

A B M Muntasir Rahman · Department of CSE, BRAC University





## PART 1

# Why Look Beyond Convolution

The three priors that made CNNs work are also three restrictions. Understanding what they cost is the motivation for everything that follows.

# What a convolution assumes

From the CNN lecture: a convolutional layer is a dense layer with three constraints imposed on it. Those constraints are hypotheses about images, and they are excellent hypotheses — they are why CNNs work with modest data. But **a hypothesis that helps you learn from little data also limits what you can learn from a lot of it**. That trade is the subject of this lecture.

## Locality

*A unit sees a  $k \times k$  patch.*

---

### The cost

A relationship between two distant pixels cannot be expressed in one layer. It must be assembled across many layers, if at all.

## Weight sharing

*One kernel applies everywhere.*

---

### The cost

The same filter is used on sky and on faces. The layer cannot treat a position differently based on what is at that position.

## Hierarchy

*Depth builds abstraction.*

---

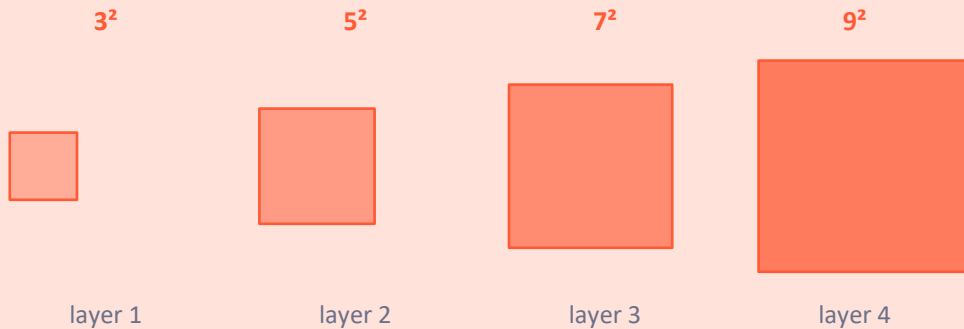
### The cost

Global context is only available deep in the network, at low resolution, after many lossy downsampling steps.

**The question this lecture answers:** what would a vision architecture look like if we removed these constraints and let the network decide, for every position, which other positions matter?

# Local versus global receptive field

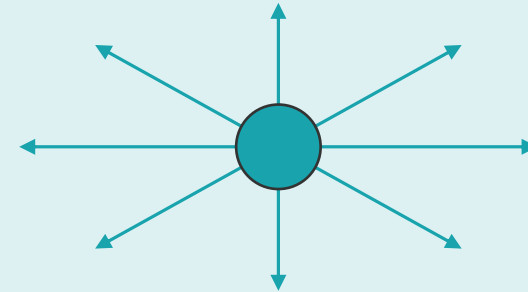
## Convolution: grow it slowly



Each  $3 \times 3$  layer adds two pixels to the receptive field. To relate two corners of a  $224 \times 224$  image, a stack of  $3 \times 3$  convolutions needs roughly 110 layers — which is why real networks downsample instead, and pay for the context in lost resolution.

**Information travels a bounded distance per layer.**

## Attention: global from layer one



Every position can read from every other position in a single operation. There is no receptive field to grow, because there is no locality constraint to overcome in the first place.

**Information travels anywhere, in one step.**

**But nothing is free.** Removing locality means the model no longer knows that neighbouring pixels are related — it has to learn that from data. It also means every position interacts with every other, so cost grows with the square of the number of positions. Parts 4 and 5 are largely about paying those two bills.

# When distance actually matters

Concrete cases where the useful evidence is nowhere near the pixel being labelled:



## Scene context disambiguates

A small white rectangle is a boat if the surrounding pixels are water and a van if they are road. Deciding needs evidence from far outside any local window.



## Symmetry and repetition

Two windows on opposite sides of a building, two wings of an aircraft, the paired organs in a scan. The relationship is between distant regions, and it is highly informative.



## Counting and layout

How many people are in the frame, or which object is in front — these are global properties of the whole set of positions, not of any neighbourhood.



## Occlusion and continuation

A cat behind a railing appears as disconnected stripes. Recognising that they belong to one animal requires linking regions that are not adjacent.



## Fine-grained comparison

Distinguishing two bird species may hinge on comparing beak shape with tail pattern — two small regions at opposite ends of the object.



## Text and structure in images

Reading a document, a chart or a road sign needs relationships between symbols across the whole image, in a specific order.

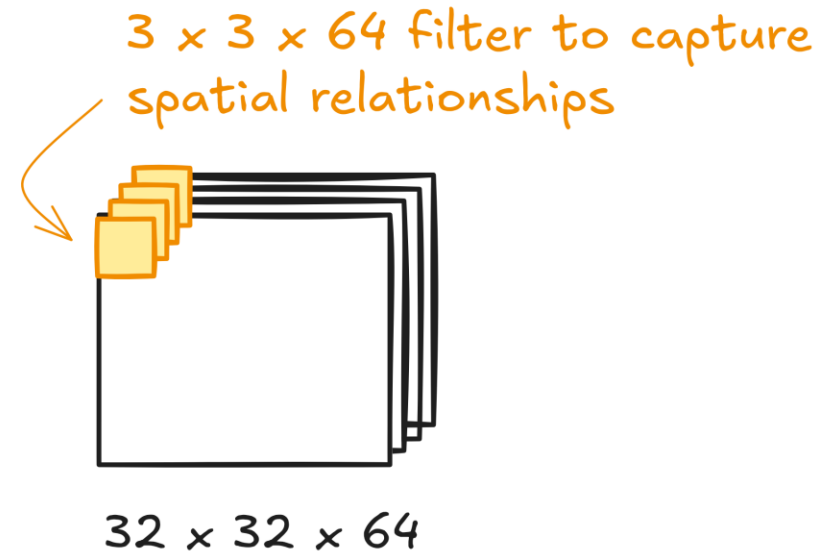
A CNN can handle all of these — with enough depth, enough downsampling, and enough data. **The argument for attention is not that convolution cannot, but that attention can do it in one layer instead of thirty, and can decide per input which relationships to use.**

# Why ViT?

## Inductive biases of convolution networks

### 1. Locality

Convolution Networks explicitly use 2D convolutions to learn these relationships.



# Why ViT?

## Inductive biases of convolution networks

### 2. Translation Equivariance

Objects shifted in an image should be recognized as the same object.

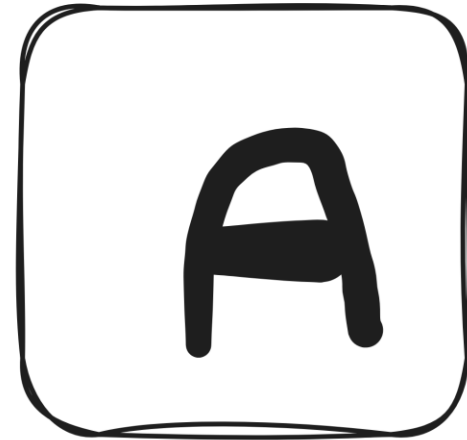
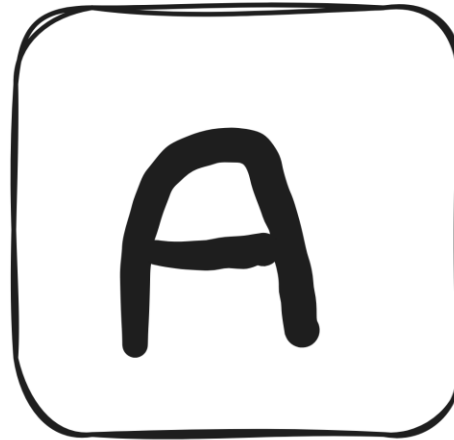


## Why ViT?

### Inductive biases of convolution networks

#### 3. High Frequency Variations

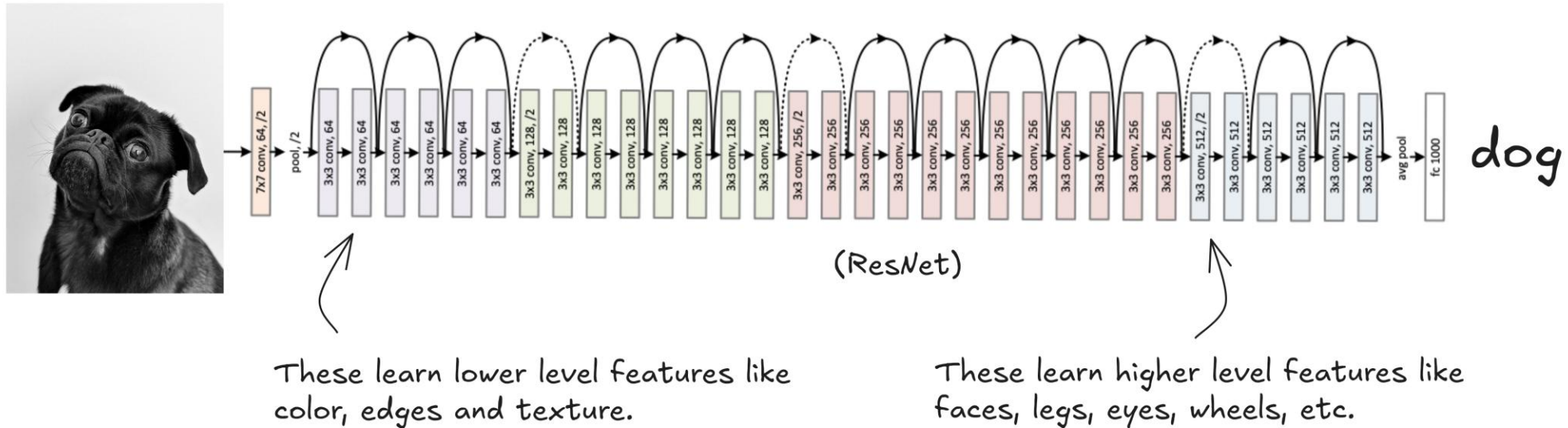
Small pixel distortions in the image should represent the same object conceptually.



# Why ViT?

## Inductive biases of convolution networks

### 4. Hierarchical feature learning



## Why ViT?

Because of these inductive biases, ResNets are more data efficient and hence outperforms ViT when trained on image classification.

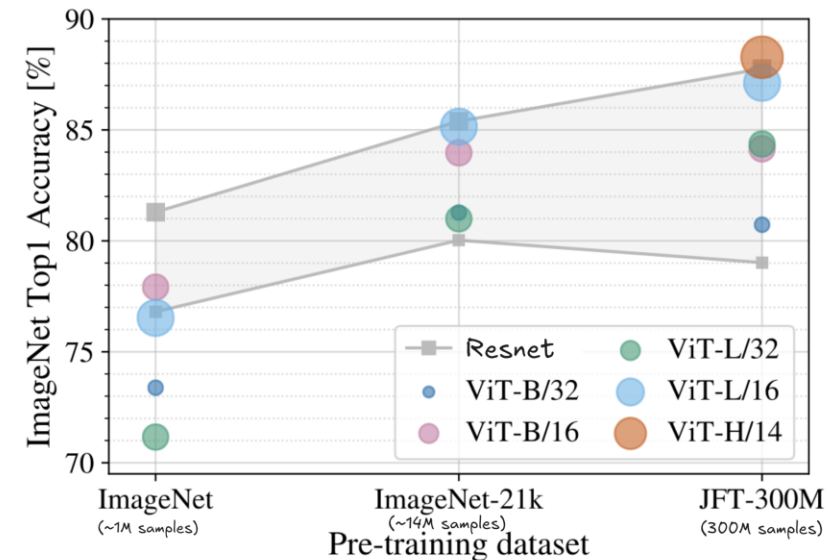
At least this is the case for moderately sized datasets like ImageNet-1K with 1.3M samples.

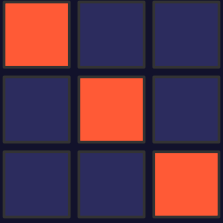
## Why ViT?

But this changes when scaling the pretraining dataset to 14M-300M samples.

Scaling the training data of ViT outweighs the inductive biases of the convolution networks!

This is why we talk about ViT





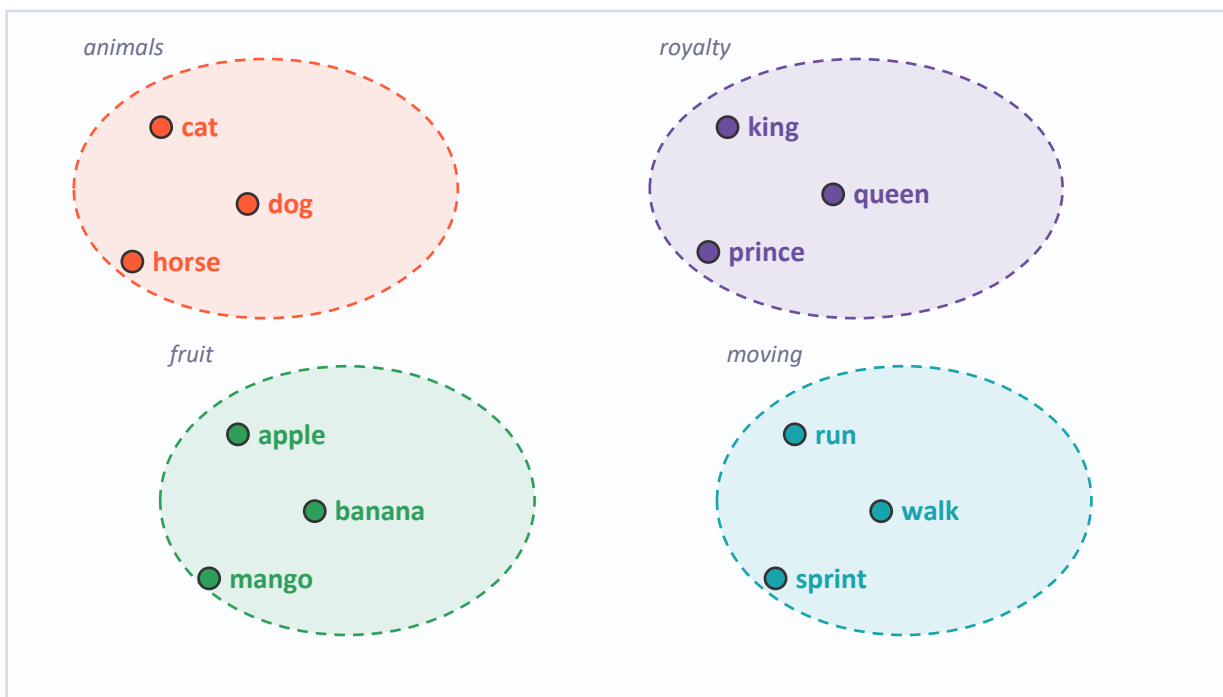
## PART 2

# Embeddings

First, what it means to represent something as a vector. No images in this part — the idea is easier to see on a general set of vectors.

# What is an embedding?

A network cannot consume a word, or a patch of pixels, or a user. It consumes vectors of numbers. **An embedding is a learned map from an item to a point in a continuous d-dimensional space** — and the whole reason it is useful is what the geometry of that space comes to mean: **items that behave similarly end up near each other.**



*A 2-D cartoon of a space that is really 768-dimensional. Nobody assigned these regions — they emerged from training.*

## Where do the numbers come from?

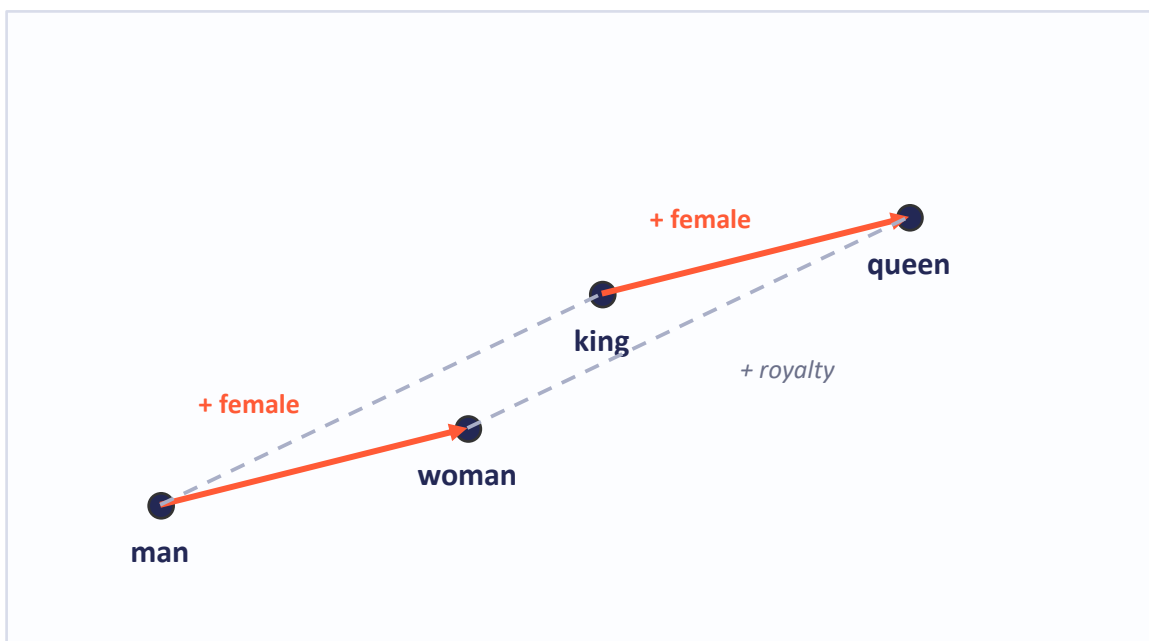
They are parameters. The embedding of "cat" starts as random noise and is updated by gradient descent like any other weight. Because the training signal depends on how the item is used, items used in similar ways receive similar vectors. The structure is a by-product of the task, not a design.

## Why this matters for the next hour

- **Similarity becomes geometry.** Asking "how related are these two things" turns into "how aligned are these two vectors", which is a computation a network can do and differentiate.
- **Everything becomes comparable.** Words, patches, audio frames and time steps all become vectors of the same width, so one mechanism can process any of them.
- **Attention is built entirely on this.** Every attention score you will see is a comparison between two embeddings.

# Directions in the space carry meaning

The striking property of a well-trained embedding space is that it is not just clustered — it is **structured**. Consistent relationships between items show up as consistent displacement vectors, the same direction and roughly the same length wherever that relationship holds.



## Other directions people have found

singular → plural · present → past tense · country → capital city ·  
adjective → comparative  
and, in image models: object → the same object rotated, or a scene →  
the same scene at night.

## An honest caveat

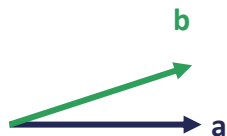
The parallelogram picture is a genuine effect but a tidied-up one. Real analogies hold approximately, the standard demonstrations exclude the query words from the answer, and many relationships have no clean direction at all. Treat it as intuition, not as a property you can rely on.

**Where this is heading.** If meaning lives in directions, then the question "does this vector point the same way as that one" is a meaningful question about content. There is a standard operation that answers it in one number, and it is the operation the entire attention mechanism is built on. That is the next slide.

# The dot product: agreement in one number

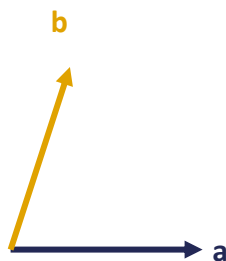
$$\mathbf{a} \cdot \mathbf{b} = \sum a_i b_i = |\mathbf{a}| |\mathbf{b}| \cos \theta$$

Two readings of the same quantity. The left is how you compute it; the right is what it means. Holding the lengths fixed, the dot product is simply a measure of how aligned two vectors are.



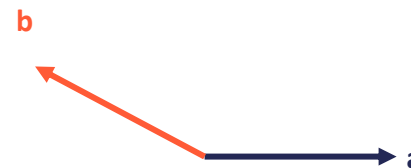
$$\mathbf{a} \cdot \mathbf{b} = +0.95$$

nearly the same direction



$$\mathbf{a} \cdot \mathbf{b} = +0.31$$

largely unrelated



$$\mathbf{a} \cdot \mathbf{b} = -0.88$$

pointing away

## Sign

+ aligned

0 orthogonal,  
unrelated

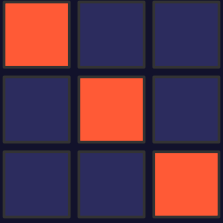
- opposed

## This is the attention score

When a token asks "how relevant is that other token to me", the answer is the dot product of its query vector with the other token's key vector. Nothing more elaborate is going on — the mechanism you are about to meet is this operation, done for every pair, and then normalised.

## Two details that matter later

Length is not ignored. A long vector produces large dot products with everything, which is one reason LayerNorm sits in front of the attention. Dimension inflates it. Summing over more components makes the typical magnitude grow with  $\sqrt{d}$  — which is exactly what the scaling factor later corrects.

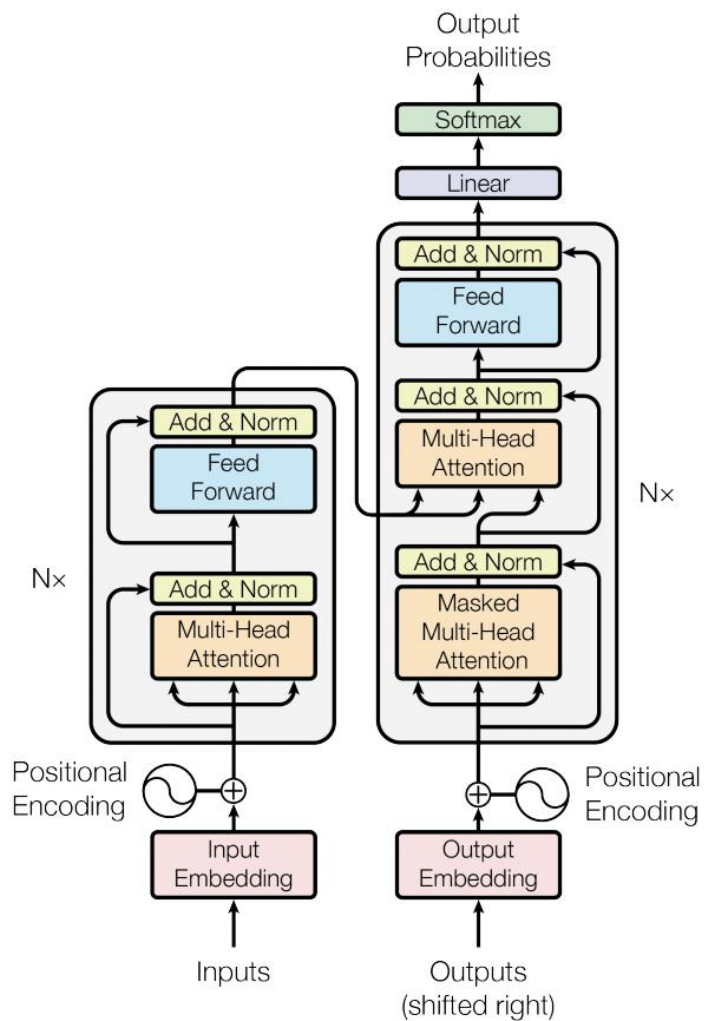


### PART 3

# The Transformer Block

Attention is one line of a larger block. The rest — normalisation, residuals, an MLP, and a new activation — is what makes it trainable, and most of it will be new to you.

# Transformer



We will see:

- Self-attention
- Multi-head attention
- Positional encoding
- Byte-pair encoding

Image Courtesy: Attention is all you need

# High level view of a transformer

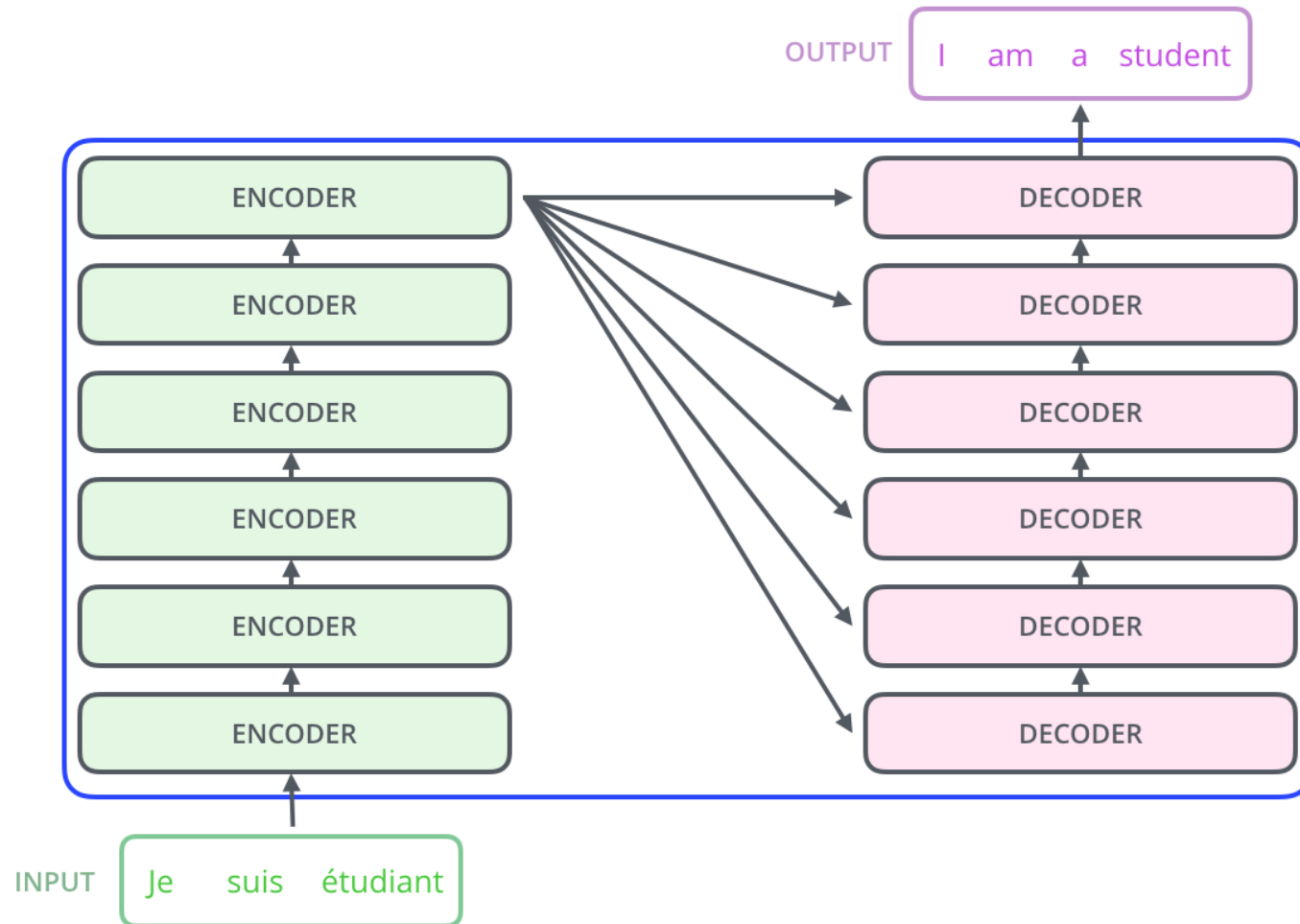


Image Courtesy: Jay Alammam

# What is Attention?

Attention is a mechanism that assigns importance weights to different tokens when computing a representation for a given token.

In simple terms:

Each token asks: “Which other tokens should I pay attention to, and how much?”

## When was attention first introduced?

Attention was first introduced in 2014 in the context of neural machine translation.

- Paper: “Neural Machine Translation by Jointly Learning to Align and Translate”
- Context :
  - Used with RNN-based encoder-decoder models
  - Solved the fixed-length bottleneck problem of seq2seq models
  - Allowed the decoder to focus on different parts of the input sequence dynamically

# Self attention

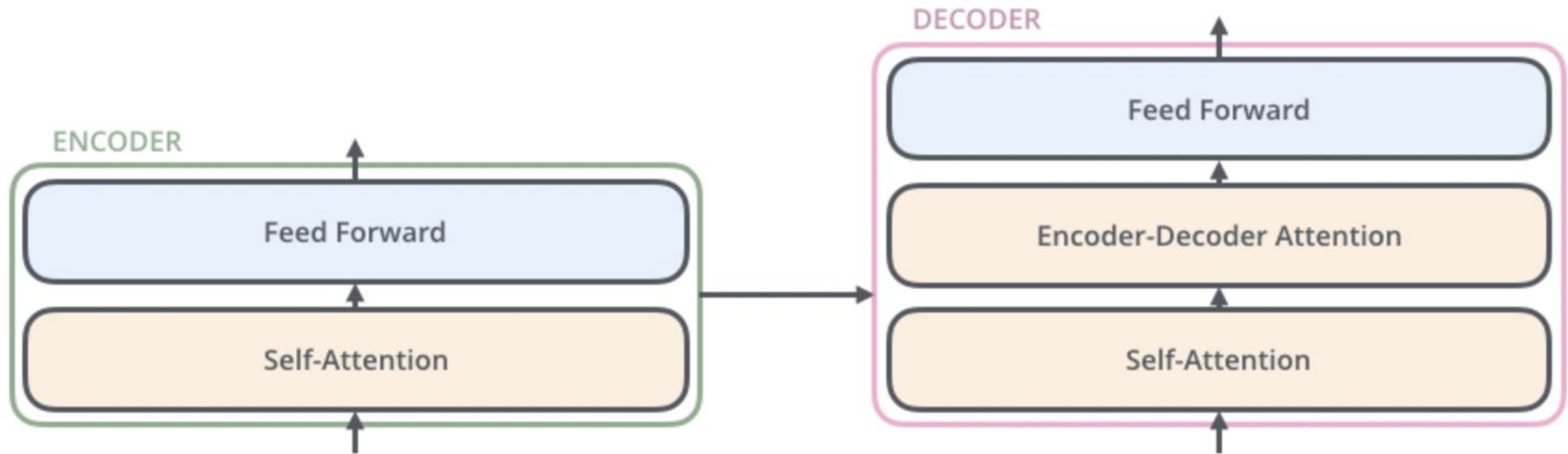


Image Courtesy: Jay Alammar

# Self attention

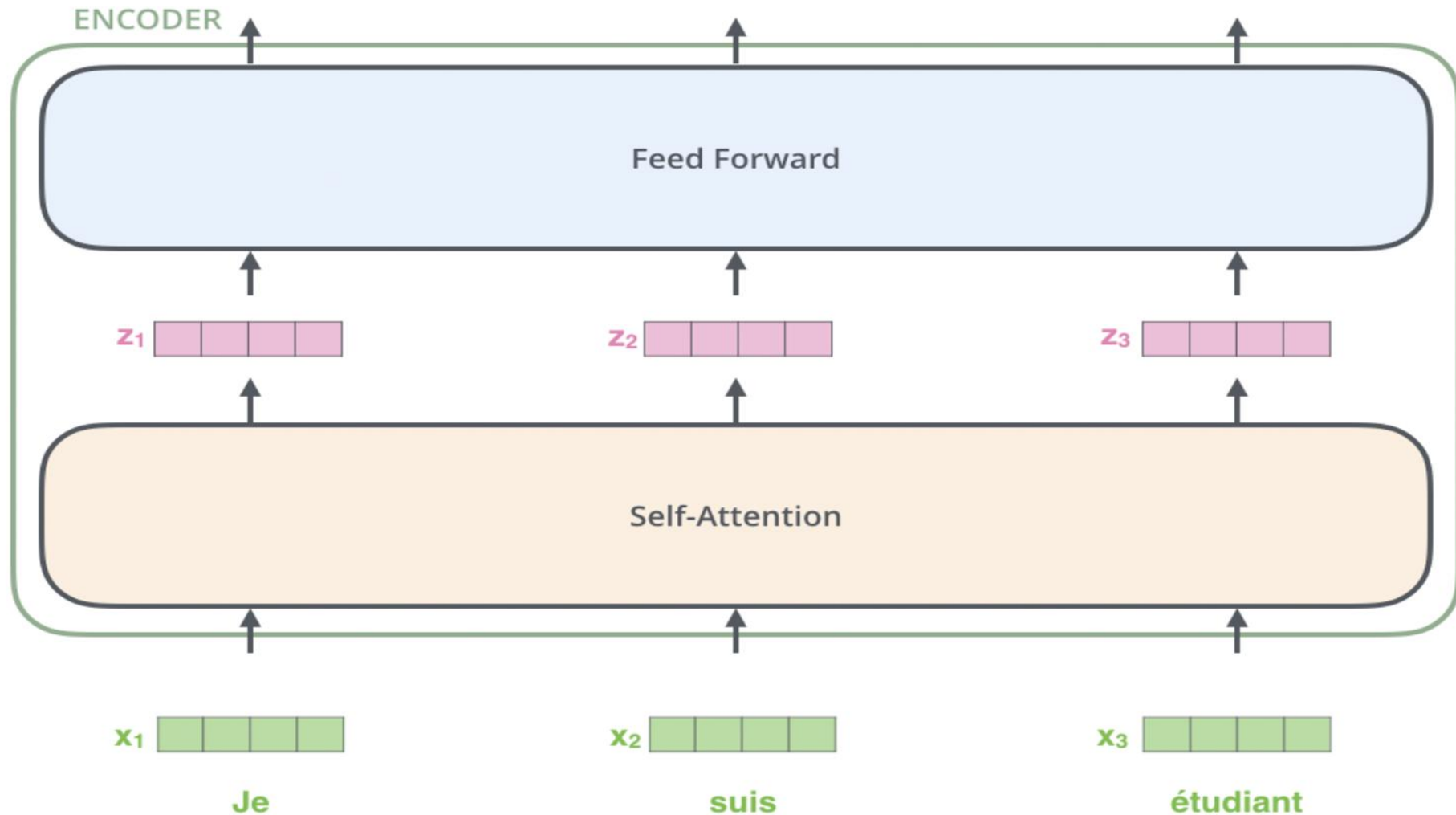


Image Courtesy: Jay Alammar

Self attention at a high level

The animal didn't cross the street because it was too tired.

It → animal? street?

For a human, pretty easy, for a machine, not so much.

# Why is it called self attention?

- Simple attention focuses on context based on a query
  - For MT, the query could be the translation task
- Self attention takes the relationship of the words in the same sentence into account

# Self attention

- An input goes through three linear transformations (mapping and resizing)
  - Query
  - Key
  - Value
- These idea came from information retrieval
  - Query: the string you search for (youtube search string)
  - Key: the string an info retrieval model matches your query with (video titles)
  - Value: the result presented to you (actual videos)
- The model returns (ranks) the best video based on the similarity of your query and the key it searches from– exactly what we want in self-attention
  - Rank the words (find their values) using your current word's representation (query) and all other words' representations (key)

# Self attention internally

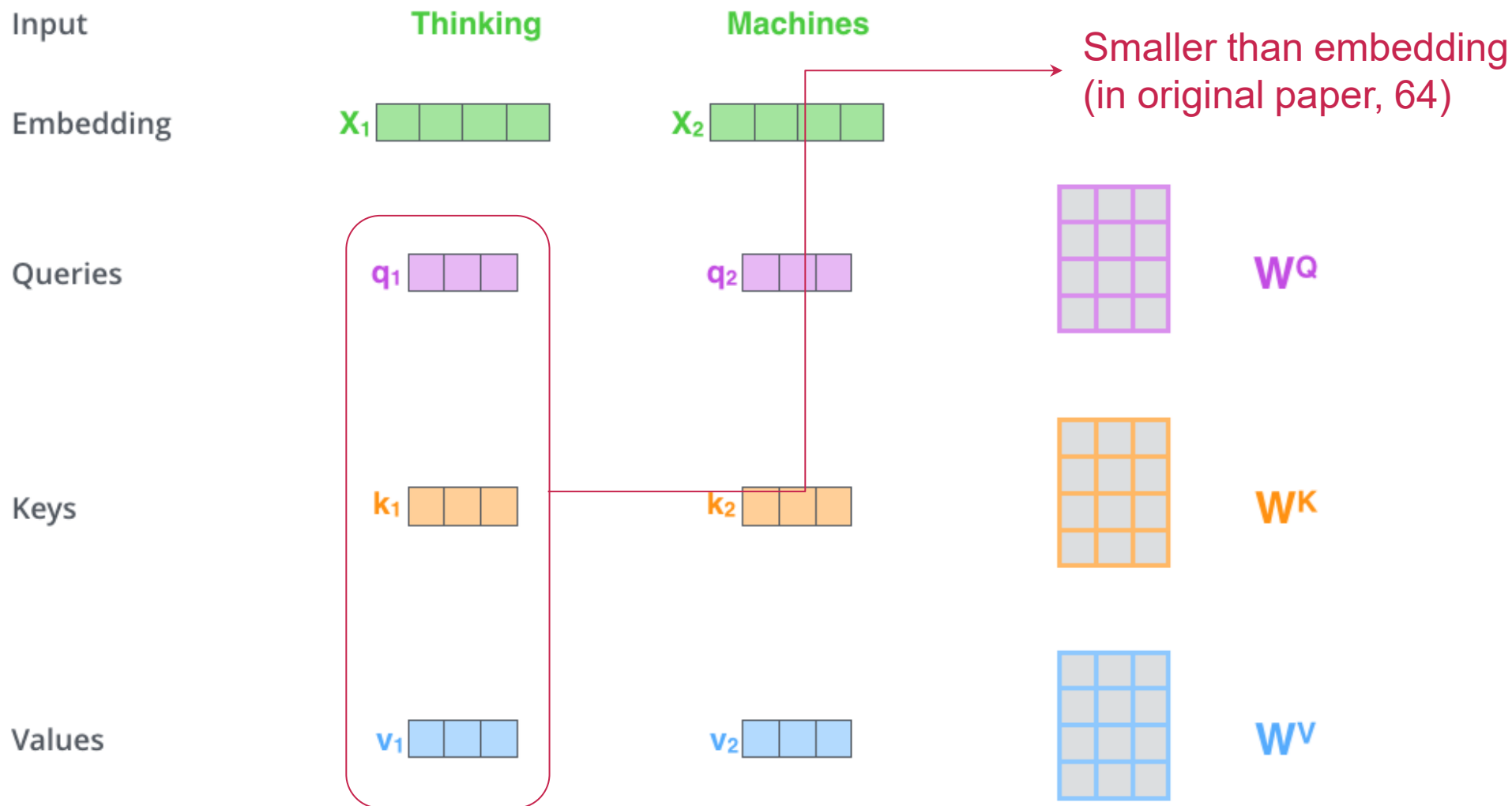


Image Courtesy: Jay Alammar

# Self attention maths

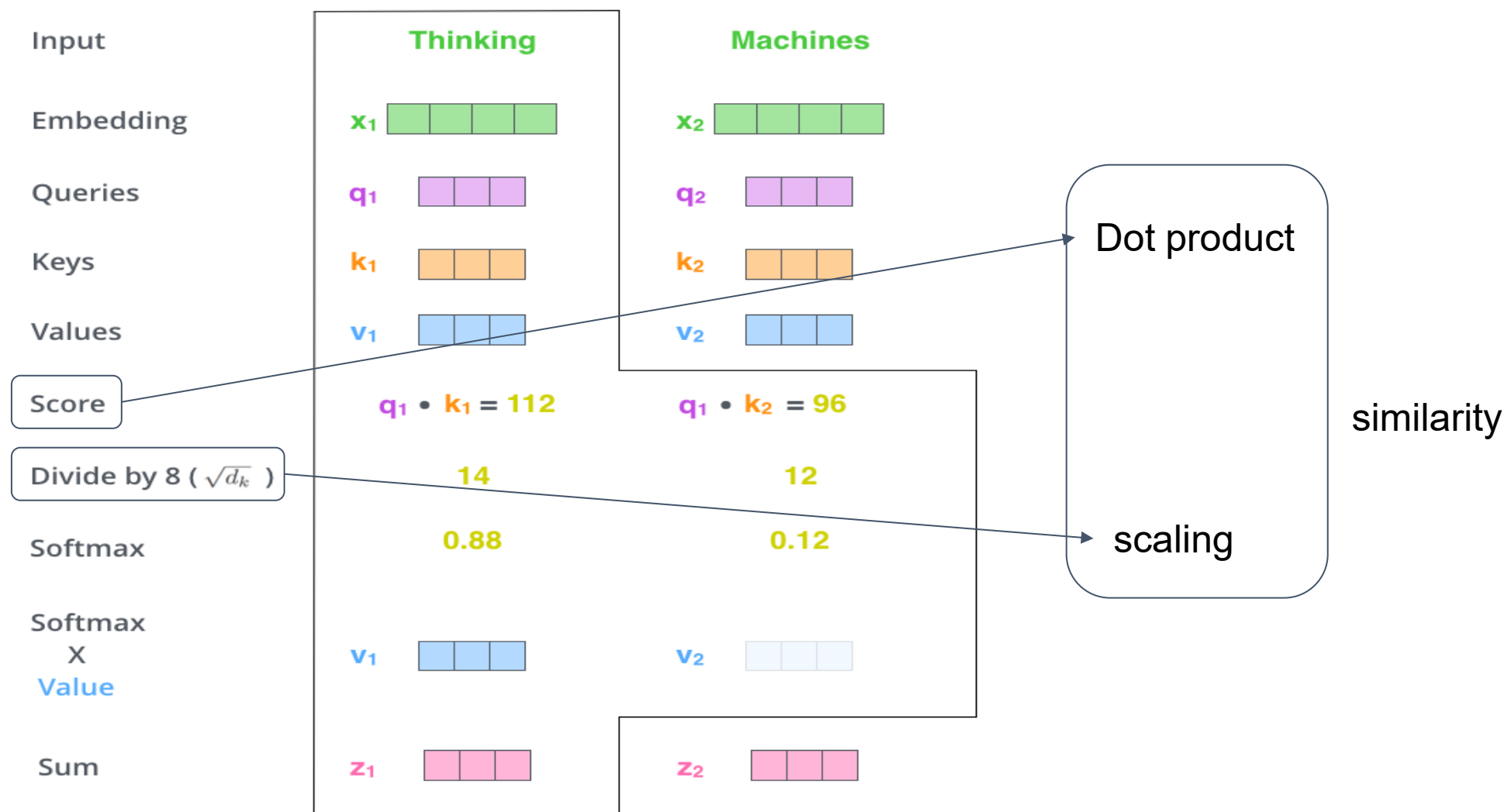
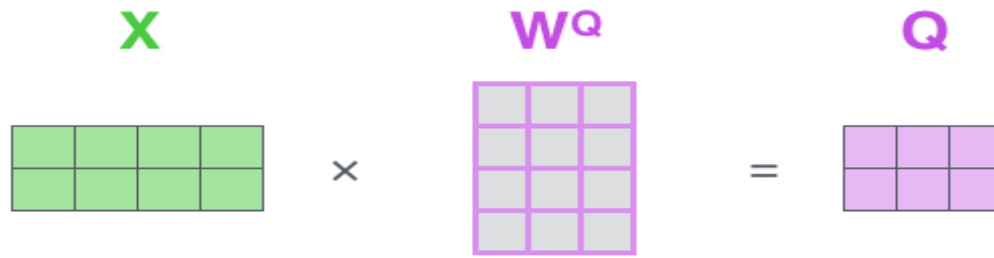


Image Courtesy: Jay Alammar

# Matrix Calculation of Self attention

$$\mathbf{X} \times \mathbf{W}^Q = \mathbf{Q}$$


The diagram illustrates the calculation of the Query matrix  $\mathbf{Q}$ . It shows a green input matrix  $\mathbf{X}$  (2 rows by 4 columns) multiplied by a purple weight matrix  $\mathbf{W}^Q$  (4 rows by 4 columns) to produce a purple output matrix  $\mathbf{Q}$  (2 rows by 3 columns). The multiplication is represented by a cross symbol ( $\times$ ) and an equals sign ( $=$ ).

$$\mathbf{X} \times \mathbf{W}^K = \mathbf{K}$$


The diagram illustrates the calculation of the Key matrix  $\mathbf{K}$ . It shows a green input matrix  $\mathbf{X}$  (2 rows by 4 columns) multiplied by an orange weight matrix  $\mathbf{W}^K$  (4 rows by 4 columns) to produce an orange output matrix  $\mathbf{K}$  (2 rows by 3 columns). The multiplication is represented by a cross symbol ( $\times$ ) and an equals sign ( $=$ ).

$$\mathbf{X} \times \mathbf{W}^V = \mathbf{V}$$


The diagram illustrates the calculation of the Value matrix  $\mathbf{V}$ . It shows a green input matrix  $\mathbf{X}$  (2 rows by 4 columns) multiplied by a blue weight matrix  $\mathbf{W}^V$  (4 rows by 4 columns) to produce a blue output matrix  $\mathbf{V}$  (2 rows by 3 columns). The multiplication is represented by a cross symbol ( $\times$ ) and an equals sign ( $=$ ).

Image Courtesy: Jay Alammur

## Encoder: Matrix Operations

The encoder processes input sequences using self-attention and feed-forward networks. Let's define:

$$X_{\text{input}} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad W_Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad W_K = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad W_V = \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix}$$

Compute  $Q$ ,  $K$ , and  $V$ :

$$Q = X_{\text{input}} W_Q = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

$$K = X_{\text{input}} W_K = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 3 \\ 3 & 7 \end{bmatrix}$$

$$V = X_{\text{input}} W_V = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 4 & 6 \\ 10 & 12 \end{bmatrix}$$

## Encoder: Self-Attention

Compute self-attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Step 1: Compute  $QK^T$ :

$$QK^T = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 3 \\ 3 & 7 \end{bmatrix}^T = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 3 \\ 3 & 7 \end{bmatrix} = \begin{bmatrix} 7 & 17 \\ 15 & 37 \end{bmatrix}$$

Step 2: Scale and apply softmax:

$$\text{softmax}\left(\frac{QK^T}{\sqrt{2}}\right) = \begin{bmatrix} 0.02 & 0.98 \\ 0.00 & 1.00 \end{bmatrix}$$

Step 3: Multiply by  $V$ :

$$\text{Attention}(Q, K, V) = \begin{bmatrix} 0.02 & 0.98 \\ 0.00 & 1.00 \end{bmatrix} \begin{bmatrix} 4 & 6 \\ 10 & 12 \end{bmatrix} = \begin{bmatrix} 9.88 & 11.88 \\ 10.00 & 12.00 \end{bmatrix}$$

# Multi-head attention

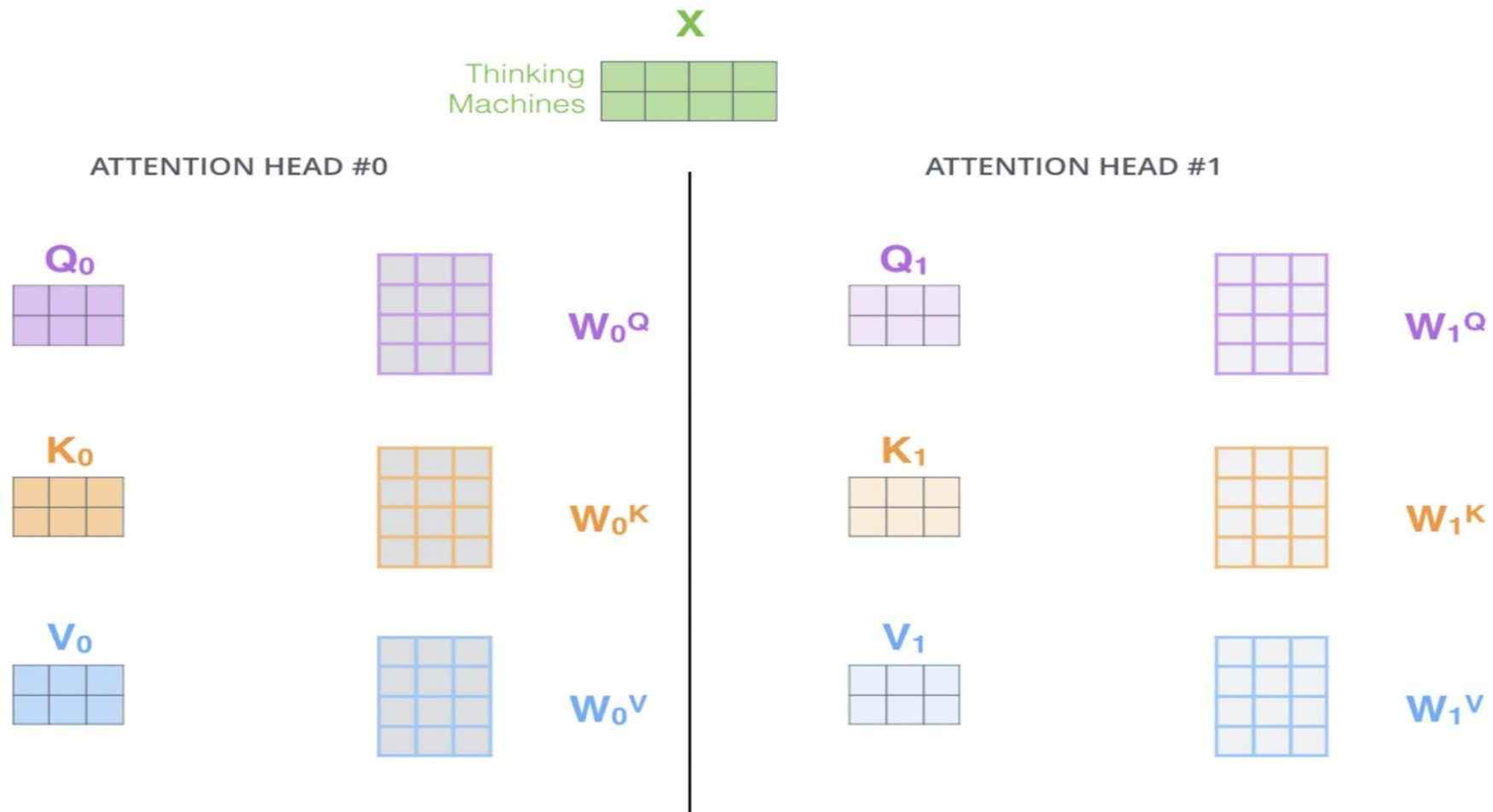
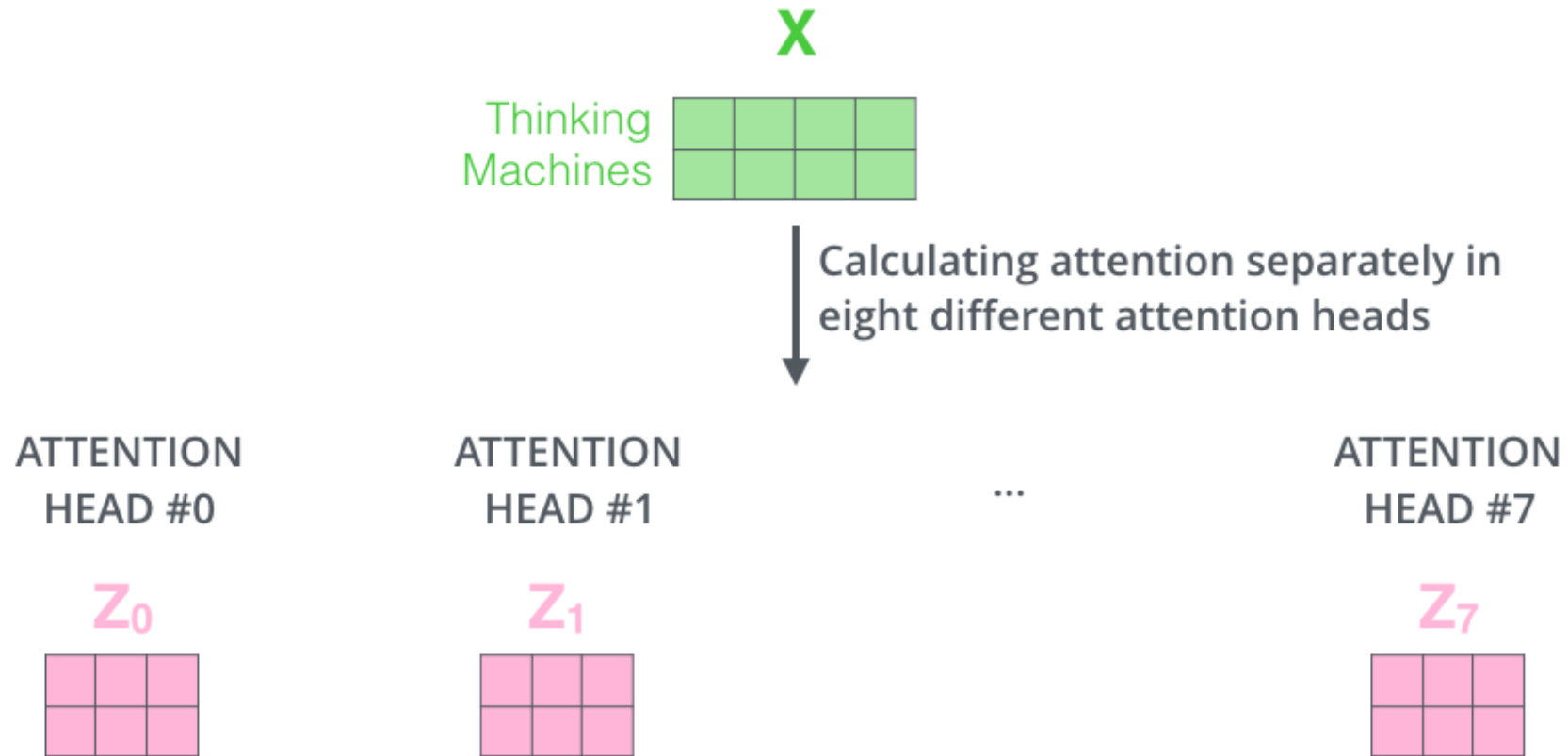


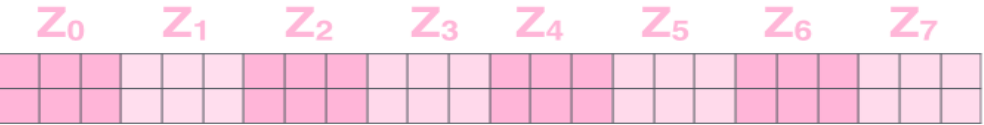
Image Courtesy: Jay Alammar

# Multi-headed attention

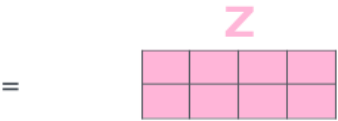


# Multi-headed attention

1) Concatenate all the attention heads

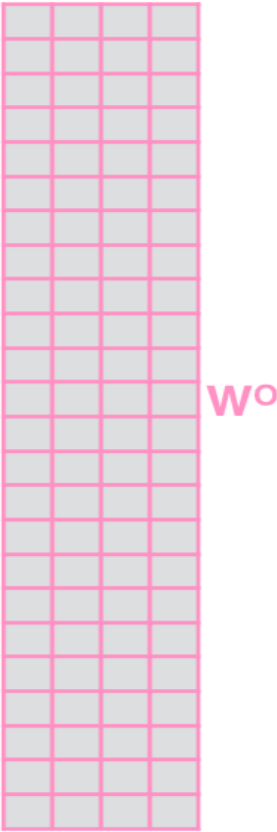


3) The result would be the  $Z$  matrix that captures information from all the attention heads. We can send this forward to the FFNN



2) Multiply with a weight matrix  $W^O$  that was trained jointly with the model

x



# Multi-headed attention

1) This is our input sentence\*

Thinking  
Machines

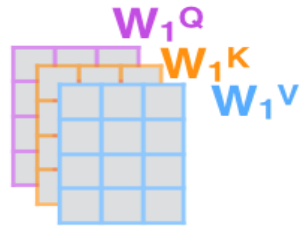
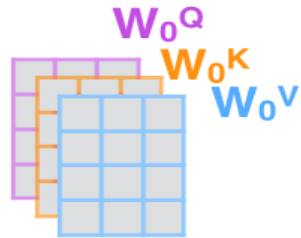
2) We embed each word\*



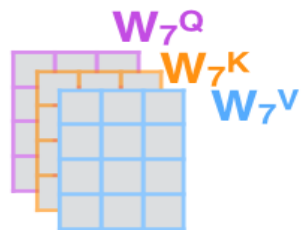
\* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one



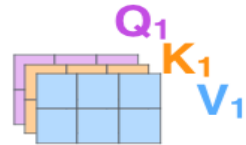
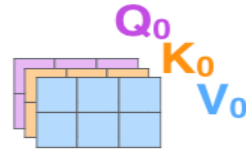
3) Split into 8 heads. We multiply  $X$  or  $R$  with weight matrices



...



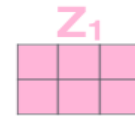
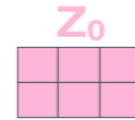
4) Calculate attention using the resulting  $Q/K/V$  matrices



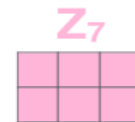
...



5) Concatenate the resulting  $Z$  matrices, then multiply with weight matrix  $W^O$  to produce the output of the layer



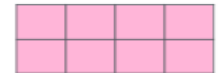
...



$W^O$



$Z$



# Why multi headed attention though?

- Because multi is better than one!
- No, really, that's it!
- Multiple heads can (hopefully) focus on more things that one head can.

## Why this even works,

Imagine you are at point A and want to go to B in a huge city.

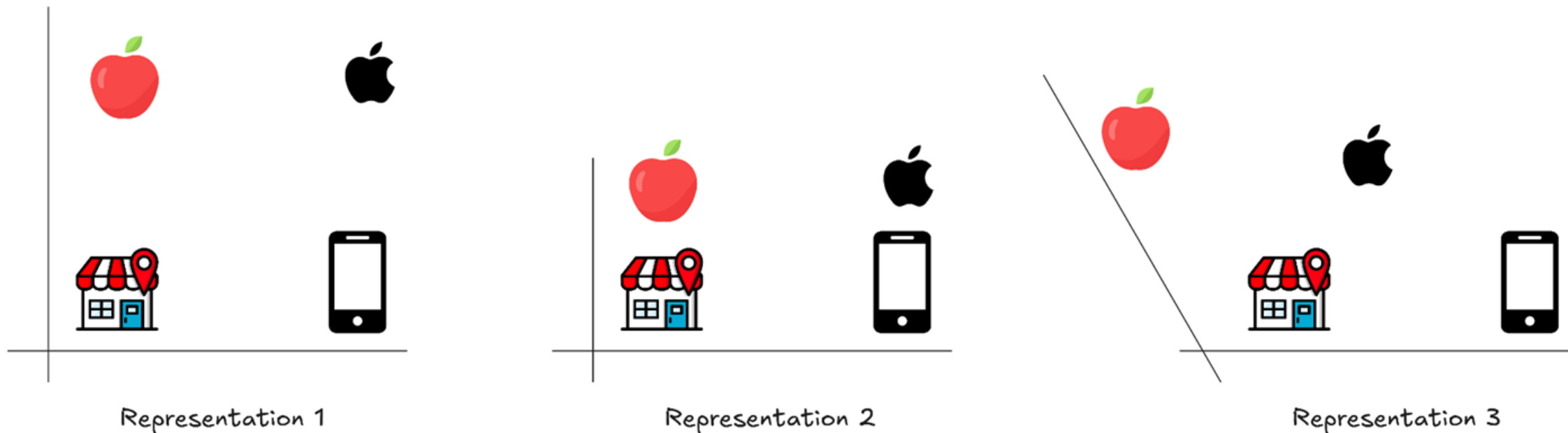
Do you think there is only one path to go there?

Of course not, there are thousands of way to reach that point.

But you will never know the best path, till you have tried a lot of them. More the better.

Hence a single matrix multiplication does not get you the best representation of query and key.

Multiple queries can be made, multiple keys can be done for each of these query. That is the reason we do so many matrix multiplications - to try and get the best key for a query that is relevant to the question asked by the user.



To visualize how Self-Attention creates different representation. Let's have a look at the three different representation of different words "apple", "market" & "cellphone".

Q : What does the company apple make?

A : Representation 2 will be best to choose for this question, and it gives us the answer "cellphone" as that is the one closest to it.

Q : Where can I get a new iphone?

A : In this case Representation 3 will be the best option, and we will get the answer "market".

# Understanding Positional Encoding

To understand What is Positional Encoding and why we need it, let's imagine the scenario in which we do not have it.

- First an input sentence, for E.g “**Pramod likes to eat pizza with his friends**”. Will be broken down into the respective words: **Pramod, likes, to, eat, pizza,with, his, friends**.
- Second, Each word will be converted into **an embedding of a given dimension**.
- Now, without Positional Encoding. The model has **no information about the relative positioning** of the words. (As everything is taken at once in parallel)
- So the sentence is no different from “**Pramod likes to eat friends with his pizza**” or any other permutation of the word.
- Hence, the reason we need PE (Positional Encoding) is **to tell the model about the positions of different words relative to each other**.

# Understanding Positional Encoding

Analogy :

Imagine you want to describe a time uniquely.

You use:

- seconds hand (changes fast)
- minutes hand (changes slower)
- hours hand (changes very slowly)

Together they uniquely identify any moment.

Positional encoding does the same:

- some dimensions change fast across positions
- some dimensions change slowly across positions

We cannot just use 0, 1, 2, 3... as a single number because that does not scale well and the model does not handle raw integers easily.

Instead we represent each position as a LIST OF SINE AND COSINE VALUES at different frequencies.

This list is the positional encoding vector.

# Positional Encoding (Optional)

| Sequence | Index<br>of token,<br>$k$ | Positional Encoding<br>Matrix with $d=4$ , $n=100$ |                               |                               |                               |
|----------|---------------------------|----------------------------------------------------|-------------------------------|-------------------------------|-------------------------------|
|          |                           | $i=0$                                              | $i=0$                         | $i=1$                         | $i=1$                         |
| I        | 0                         | $P_{00}=\sin(0)$<br>= 0                            | $P_{01}=\cos(0)$<br>= 1       | $P_{02}=\sin(0)$<br>= 0       | $P_{03}=\cos(0)$<br>= 1       |
| am       | 1                         | $P_{10}=\sin(1/1)$<br>= 0.84                       | $P_{11}=\cos(1/1)$<br>= 0.54  | $P_{12}=\sin(1/10)$<br>= 0.10 | $P_{13}=\cos(1/10)$<br>= 1.0  |
| a        | 2                         | $P_{20}=\sin(2/1)$<br>= 0.91                       | $P_{21}=\cos(2/1)$<br>= -0.42 | $P_{22}=\sin(2/10)$<br>= 0.20 | $P_{23}=\cos(2/10)$<br>= 0.98 |
| Robot    | 3                         | $P_{30}=\sin(3/1)$<br>= 0.14                       | $P_{31}=\cos(3/1)$<br>= -0.99 | $P_{32}=\sin(3/10)$<br>= 0.30 | $P_{33}=\cos(3/10)$<br>= 0.96 |

Positional Encoding Matrix for the sequence 'I am a robot'

# Residuals

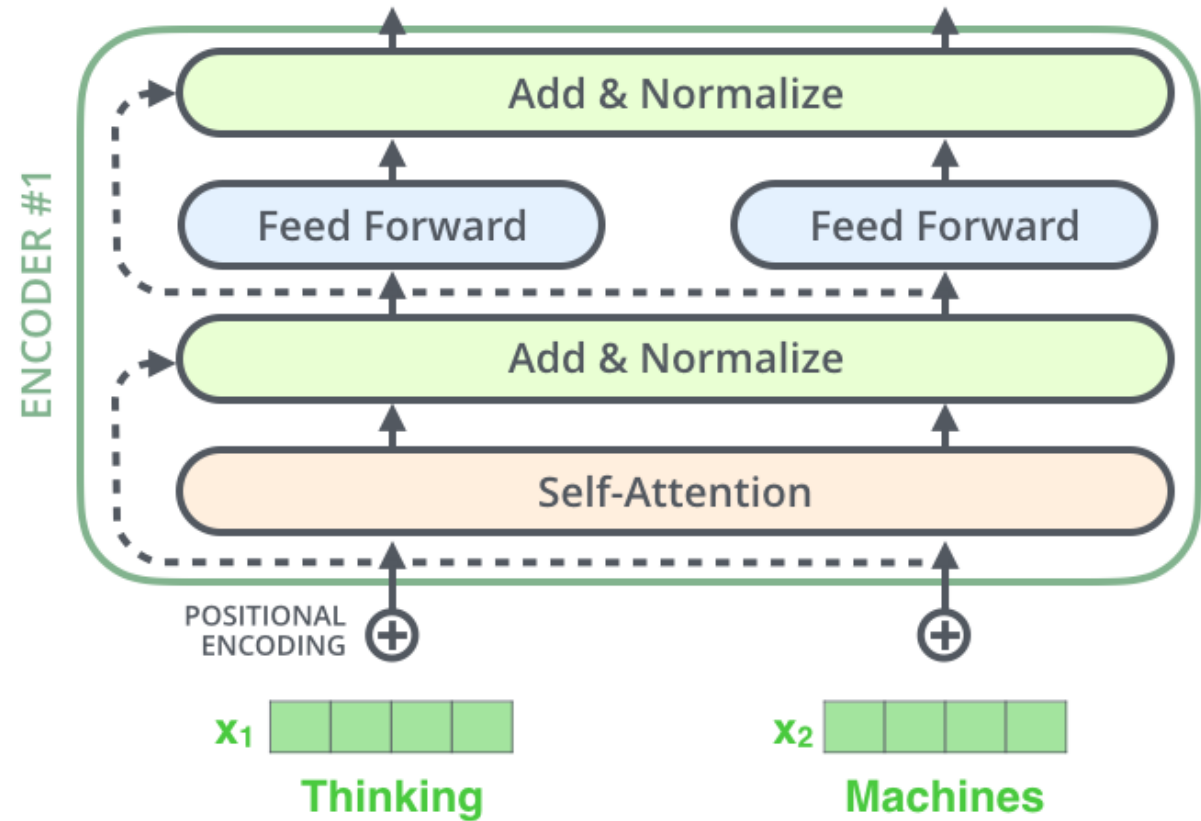
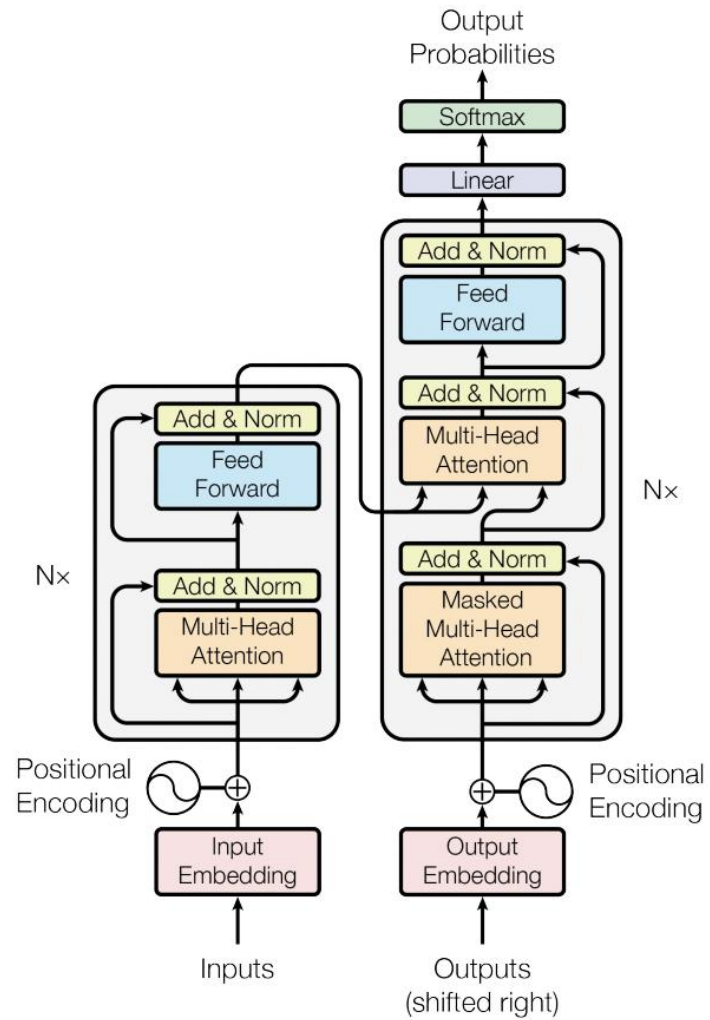


Image Courtesy: Jay Alammam

# Residuals

- Preserves knowledge and handles vanishing gradient issues
- Called Add-norm layers
- Add: literally adds the multi-headed attention outputs and the positional encoding outputs
- Norm: normalizes the outputs across axis (see skipped slide, not very important)
- That ends the multi-headed representation ordeal, now you can use these representations to actually learn a feed-forward neural network that will give you the actual encoder output

# Input embeddings

- Much simpler than the other word embeddings
  - Creates a vocabulary (needs a limited sized vocabulary, BPE handles that)
  - Assigns each word an index number and a random vector representation
  - Original paper had 512 sized vectors

# Tokenization

- Word tokenization is great, but there are more intelligent ways to tokenize
- Some languages like to combine words to create new words
  - German: Krankenhaus → Hospital (Kranken: sick, haus: house)
- English does inflections
  - Low, lower, lowest
- If we can focus on the core building words, we can reduce our vocabulary size significantly
  - Stemming and lemmatization is not going to be used

# Byte-pair encoding

- We convert words into character (byte) pairs and encode them incrementally
- Example from Wikipedia:

aaabdaaabc  $\rightarrow$  ZabdZabc (Z=aa)  $\rightarrow$  ZYdZYac (Y=ab)  $\rightarrow$  XdXac (X=ZY)

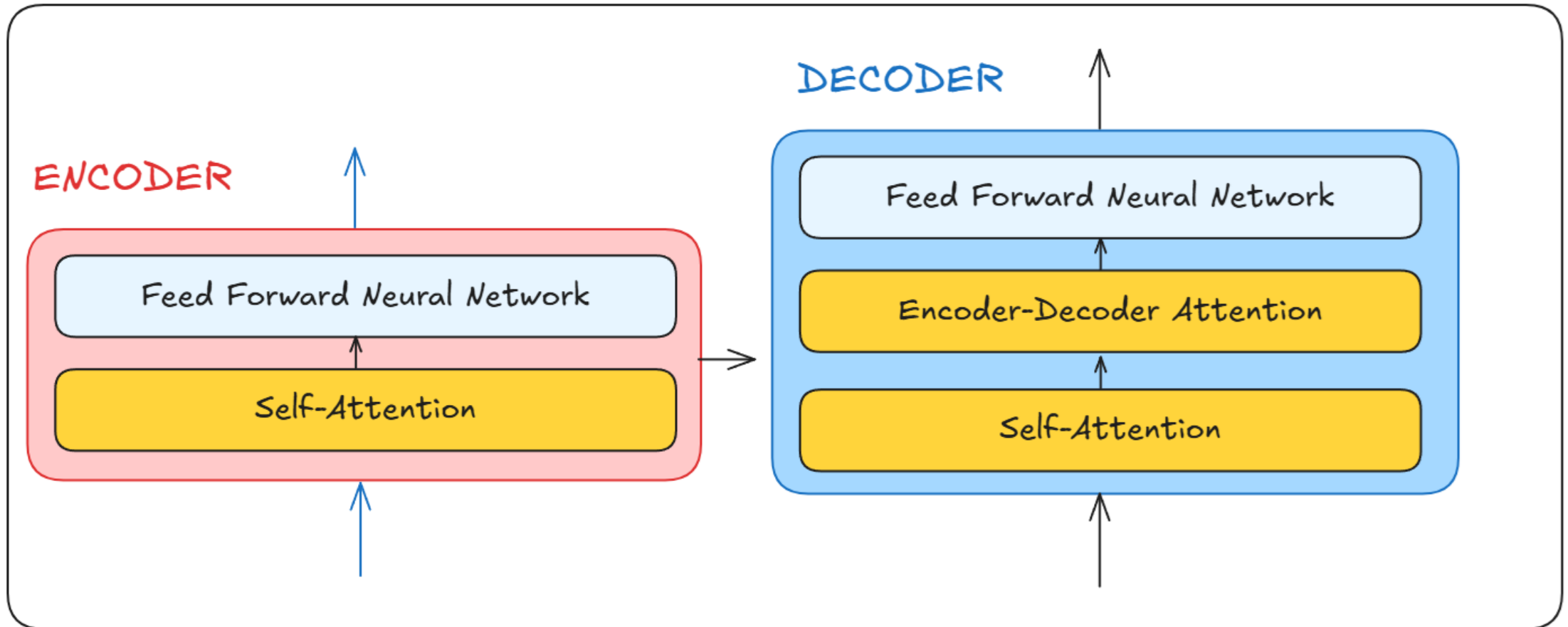
- How can we use this in transformer?  
<https://arxiv.org/pdf/1508.07909.pdf>

# Decoder Block

The decoder uses components similar to the encoder.

The key difference lies in **how these components are combined and used**.

The decoder block is **auto-regressive**. Meaning it **outputs one after the other and takes its own output as an input**.



# Decoder Block

## Encoder processing

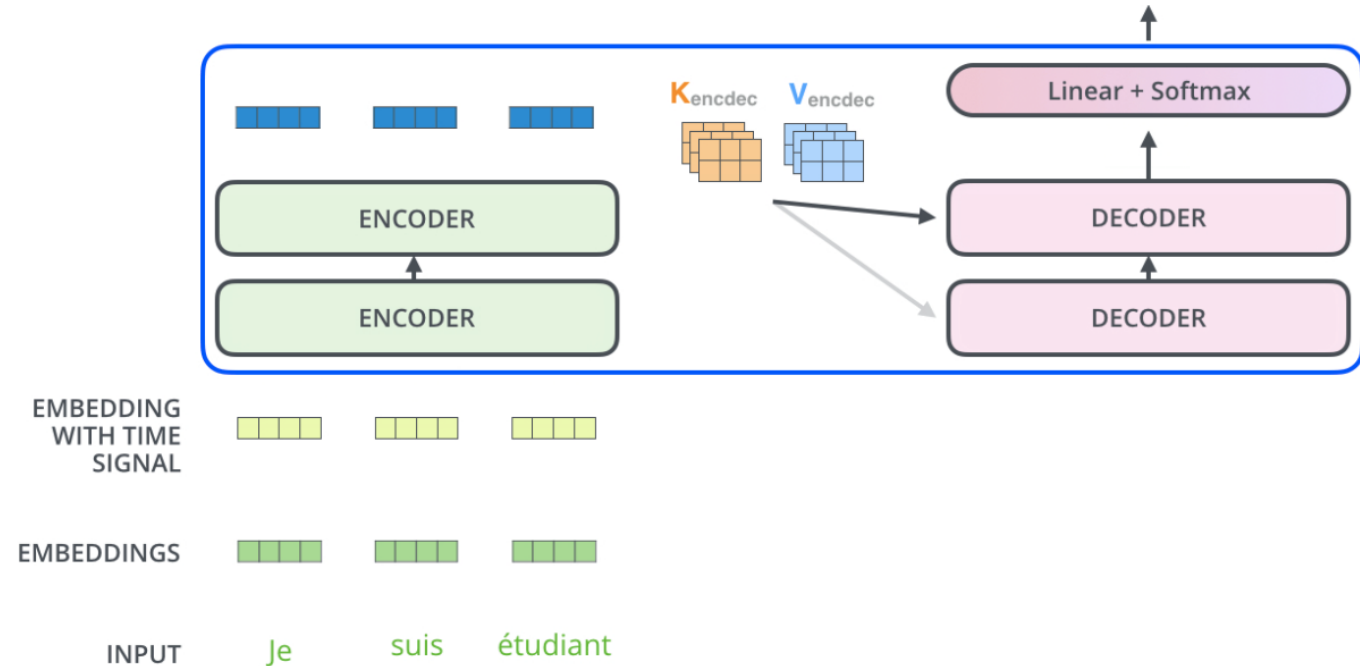
- The encoder first processes the entire input sequence
- The output of the top encoder layer contains contextual representations of the input

## Preparing encoder outputs for the decoder

- Encoder outputs are transformed into:
  - Key (K) vectors
  - Value (V) vectors
- These vectors represent the encoded information of the input sequence

## Encoder-Decoder attention in the decoder

- In this sublayer:
  - The decoder uses its own representations as Queries (Q)
  - The encoder-provided K and V guide the attention
- This allows the decoder to:
  - Focus on the most relevant parts of the input sequence
  - Align generated output tokens with input tokens



## Purpose

- Ensures the decoder generates output that is grounded in the input sequence
- Enables effective sequence-to-sequence mapping (e.g., translation)

# Decoder Block

## Autoregressive decoding loop

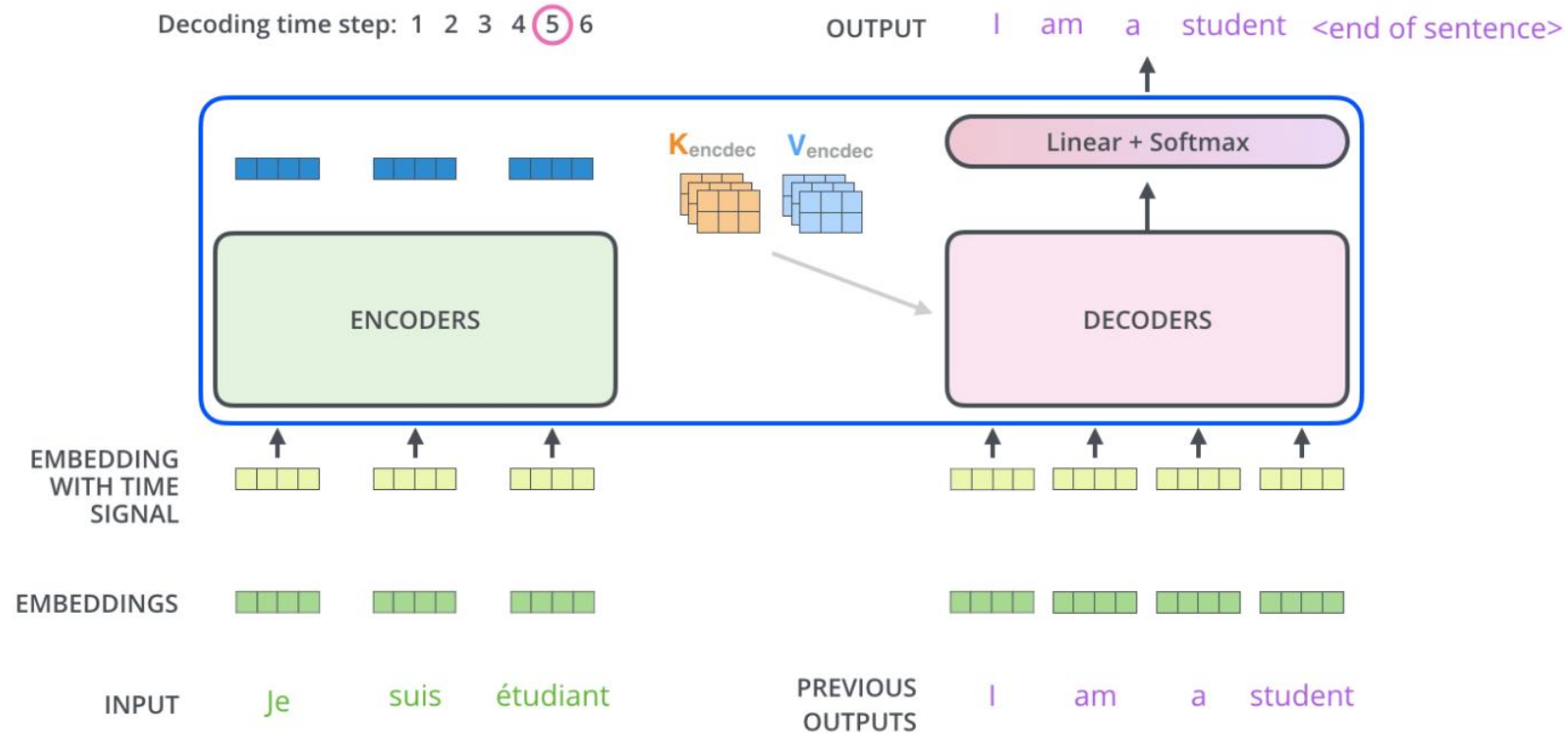
- Decoding proceeds step by step until an end-of-sequence token is generated
- This token indicates that the decoder has finished producing the output sequence

## Token feedback mechanism

- At each decoding step:
  - The newly generated token is fed back as input to the decoder
  - It is provided to the lowest decoder layer for the next time step

## Processing through the decoder stack

- The decoder processes the input token through all its stacked layers
- Intermediate decoding results propagate upward through the stack, similar to encoder processing



## Embedding and positional information

Each input token to the decoder is:

- Converted into an embedding vector
- Augmented with positional encoding
- Positional encoding ensures the decoder is aware of token order

## Overall effect

- Enables coherent, ordered, and context-aware sequence generation
- Ensures each new token is generated based on all previously produced tokens

# Masked Self-Attention in Detail

Recall in encoder self-attention, every token can attend to every other token. In the decoder, each token can only attend to itself and tokens before it. Future tokens are hidden. This is called masking.

## Why Masking?

During generation, future words do not exist yet. Allowing the model to see them during training would be cheating. The model must learn to predict the next word without peeking ahead.

## How Masking Works

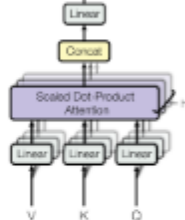
Before softmax, a mask is added to the raw attention scores. Future positions are set to  $-\infty$  so that after softmax they become 0.

## Attention mechanism

Scaled Dot-Product Attention



Multi-Head Attention

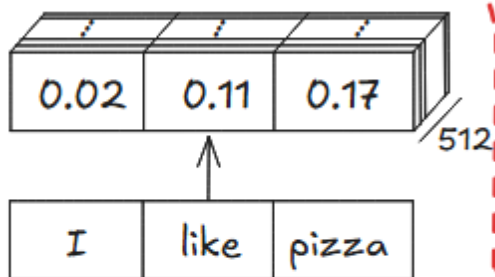


This is where the magic happens

Importance of different words are learnt relative to itself

So you can answer questions like  
What do I like?

PIZZA



Converts each word\* to an embedding of given size

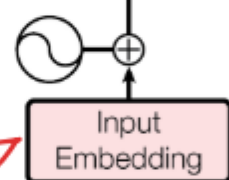
\*token will be a more appropriate term but more on that later

## Residual connections

Make it easier for the network to preserve important information from earlier layers

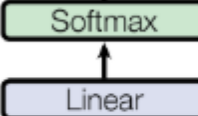
Nx

Positional Encoding



Inputs

Output Probabilities



Outputs (shifted right)

Linear layer  $\rightarrow y = xA^T + b$

ReLU  $\rightarrow \max(0, x)$

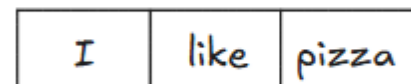
A network with sequence of linear layer, dropout and ReLU activation

$$y = \frac{x - E[x]}{\sqrt{\text{Var}[x] + \epsilon}} * \gamma + \beta$$

Layer Norm to stabilize training

Nx

1st 2nd 3rd



Adds positional information to the embeddings

This is required as the transformer processes requests in parallel

## Decoder: Matrix Operations

The decoder generates output sequences using masked self-attention and encoder-decoder attention. Let's define:

$$Y_{\text{input}} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}, \quad W_Q = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad W_K = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}, \quad W_V = \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix}$$

Compute  $Q$ ,  $K$ , and  $V$  for masked self-attention:

$$Q = Y_{\text{input}} W_Q = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$K = Y_{\text{input}} W_K = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} 5 & 11 \\ 7 & 15 \end{bmatrix}$$

$$V = Y_{\text{input}} W_V = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 16 & 18 \\ 22 & 24 \end{bmatrix}$$

# Decoder: Masked Self-Attention

Apply masking to prevent future token access:

$$\text{MaskedAttention}(Q, K, V) = \text{softmax} \left( \frac{M \odot QK^T}{\sqrt{d_k}} \right) V$$

where  $M$  is a mask matrix:

$$M = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

Step 1: Compute  $QK^T$ :

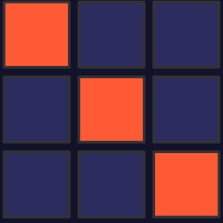
$$QK^T = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \begin{bmatrix} 5 & 11 \\ 7 & 15 \end{bmatrix}^T = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} \begin{bmatrix} 5 & 7 \\ 11 & 15 \end{bmatrix} = \begin{bmatrix} 91 & 113 \\ 113 & 141 \end{bmatrix}$$

Step 2: Apply mask and softmax:

$$M \odot QK^T = \begin{bmatrix} 91 & 0 \\ 113 & 141 \end{bmatrix}, \quad \text{softmax} \left( \frac{M \odot QK^T}{\sqrt{2}} \right) = \begin{bmatrix} 1.00 & 0.00 \\ 0.42 & 0.58 \end{bmatrix}$$

Step 3: Multiply by  $V$ :

$$\text{MaskedAttention}(Q, K, V) = \begin{bmatrix} 1.00 & 0.00 \\ 0.42 & 0.58 \end{bmatrix} \begin{bmatrix} 16 & 18 \\ 22 & 24 \end{bmatrix} = \begin{bmatrix} 16.00 & 18.00 \\ 19.48 & 21.48 \end{bmatrix}$$

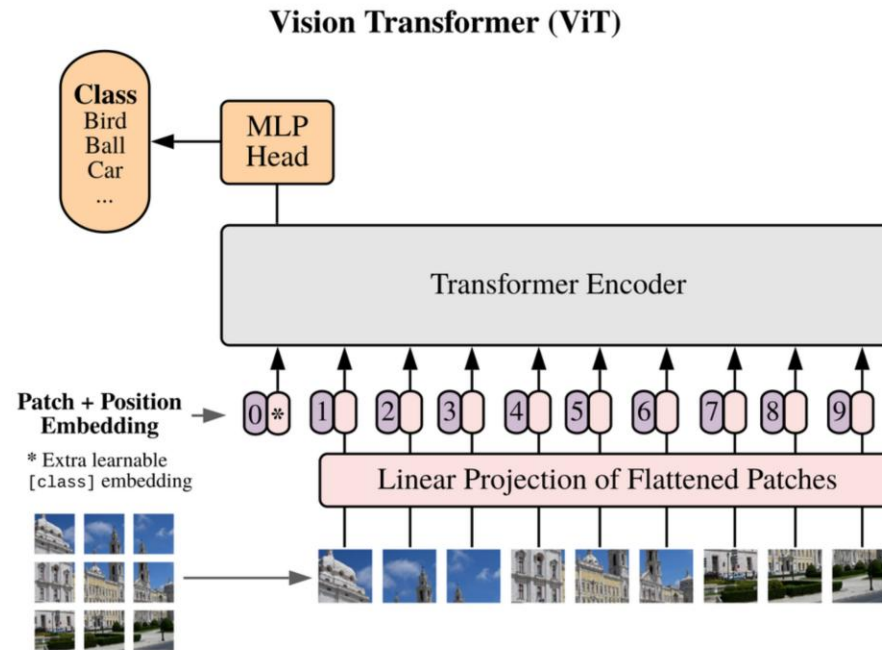


## PART 4

# The Vision Transformer

Take the block we just built, feed it image patches, and change nothing else. That is genuinely the whole of ViT.

# Vision Transformer



what, why and how?

# The [CLS] token and positional embeddings

## The [CLS] token

We need a single vector to classify, but the block outputs 196 of them. ViT borrows BERT's trick: **prepend one extra learnable token that corresponds to no patch at all**. It participates in every attention operation like any other token, so after L blocks it has aggregated information from the whole image. Its final state is fed to a linear classifier; the 196 patch outputs are discarded.

## Or just average the tokens

Global average pooling over the 196 patch outputs works just as well — and in several follow-up studies slightly better — provided you retune the learning rate.

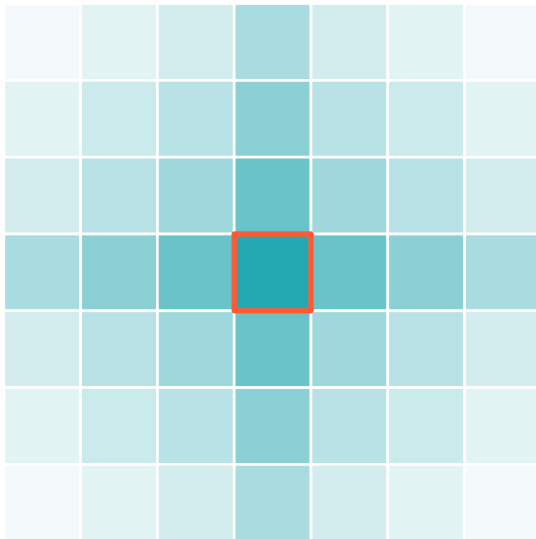
That is a useful piece of evidence: the [CLS] token is an inheritance from language modelling rather than a necessity. Swin, SegFormer and most later vision transformers drop it entirely and pool instead.

## Positional embeddings in ViT

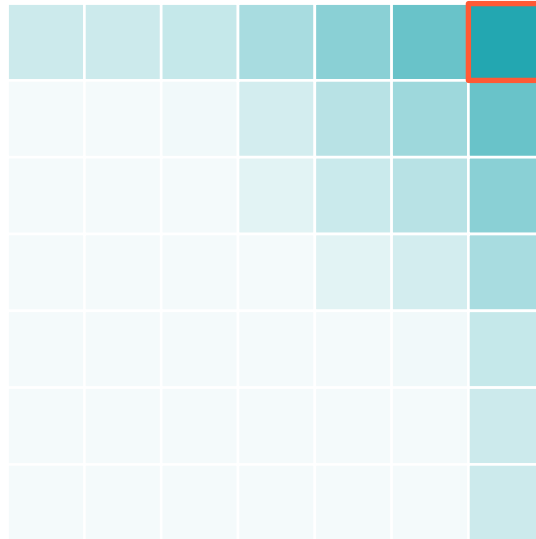
- **What ViT does.** A learned table of 197 vectors of width  $d$ , added to the tokens before the first block. Nothing tells the model that patch 15 sits directly below patch 1 — it has to discover the grid layout from data. Remarkably, trained models do: visualising the learned embeddings shows they arrange themselves into a 2D grid of similarity, recovering the spatial structure on their own.
- **The resolution problem.** Change the input size and the number of patches changes, so the table no longer fits. Standard practice is to reshape the position embeddings to a 2D grid and bicubically interpolate them to the new grid size. This works, but is a real nuisance and a common source of subtle accuracy loss when fine-tuning at a different resolution.
- **Better options later.** Relative position bias (Swin) adds a learned term to the attention scores based on the offset between patches, which generalises across resolutions. SegFormer removes explicit positional encoding entirely and lets a depthwise convolution inside the MLP supply position information.

# What the position embeddings actually learn

The position table is initialised randomly and nothing tells the model that patch 8 sits directly below patch 1. So it is worth asking what ends up in it. The standard diagnostic is to take one position's embedding and compute its cosine similarity with every other position's, then lay the results out on the patch grid.



query position: centre  
similarity peaks at itself and stays high along its own row and column

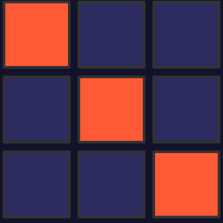


query position: top-right corner  
the same pattern, translated — the structure is relative, not memorised per position

## What this tells you

- **It recovers 2-D layout.** Nearby positions get similar embeddings, so the model has reconstructed the grid from a 1-D list of vectors.
- **Rows and columns matter.** The elevated cross shape shows the model has separately encoded horizontal and vertical position.
- **It generalises across positions.** The pattern translates with the query, which is why interpolating the table to a new resolution works at all.

**The argument this settles.** A common objection to ViT is that discarding the locality prior throws away something the model can never recover. This figure is the counter-evidence: given enough data the model rebuilds a spatial prior on its own, in a form it chose rather than one we imposed.

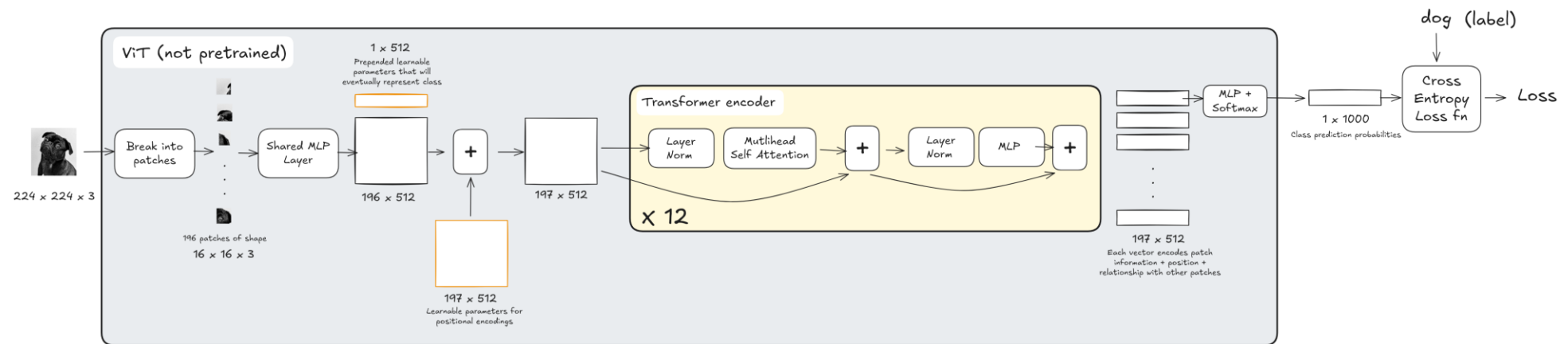


## PART 6

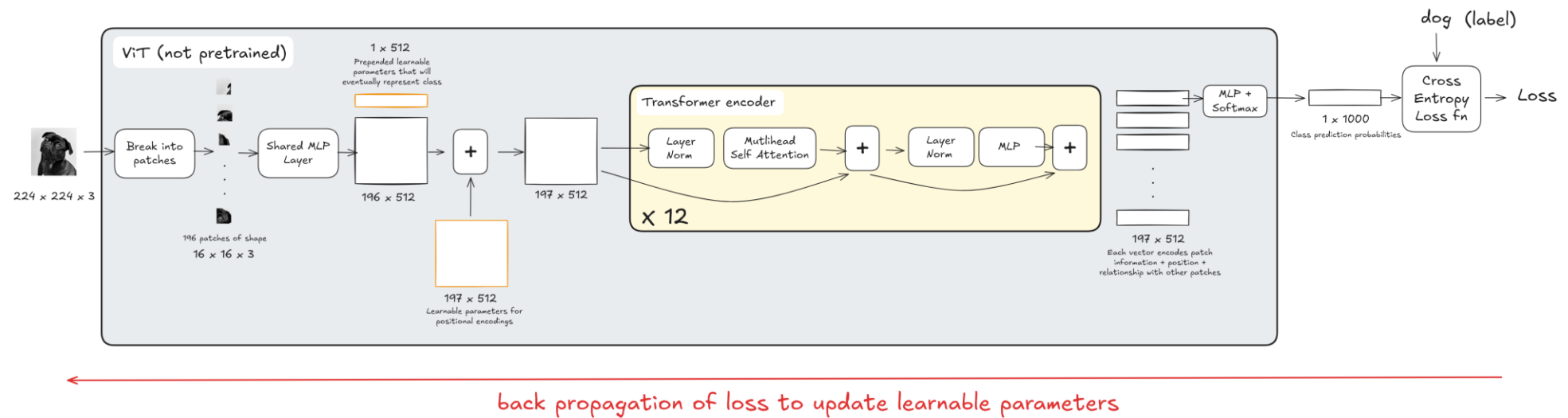
# Training and Using ViTs

Where the pretrained weights come from, how to fine-tune them, and how to decide whether you want a transformer at all.

## Pre-training ViT

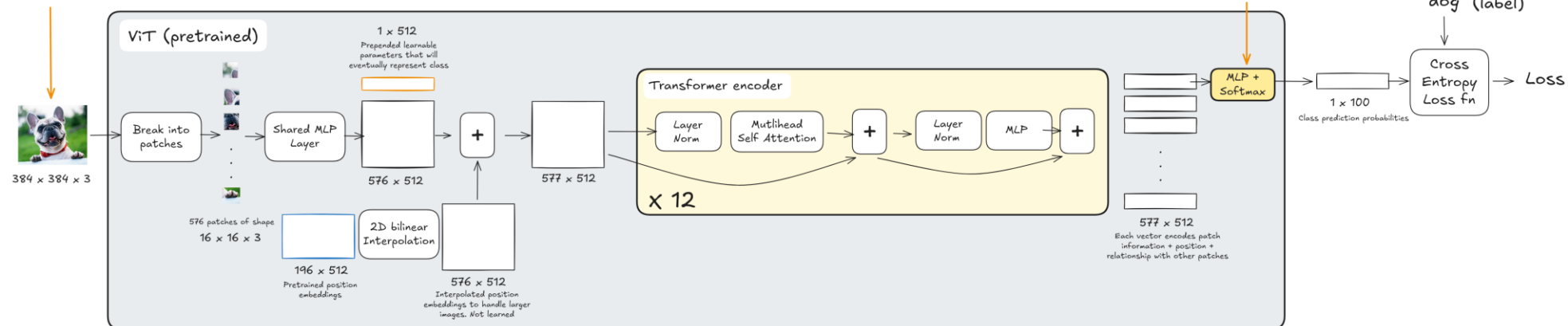


## Pre-training ViT



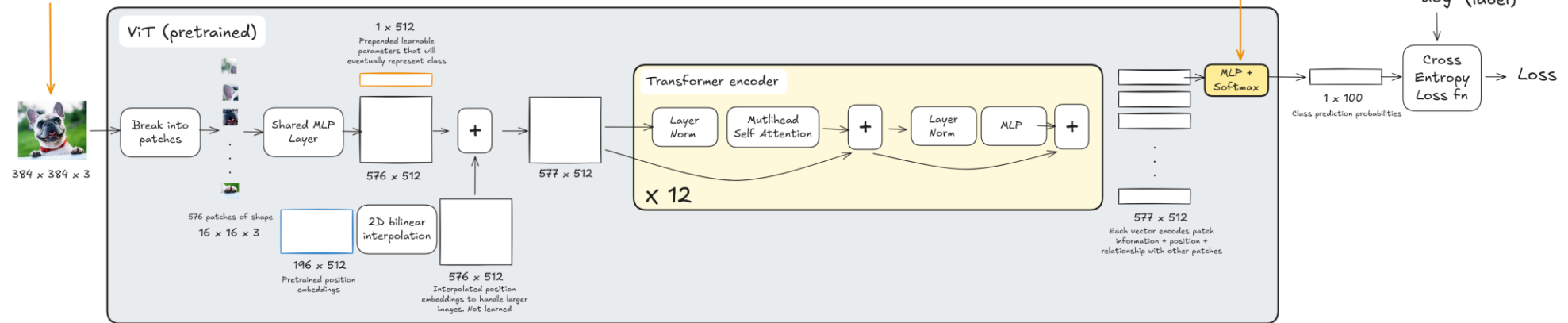
## Fine tuning ViT

Fine tuned on higher resolution image, classification pairs that are fewer in number



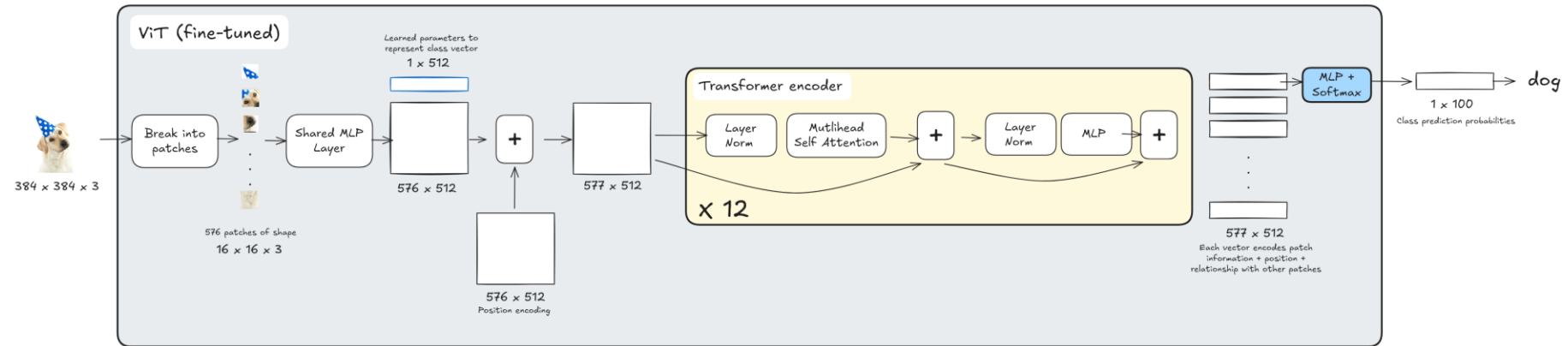
## Fine tuning ViT

fine tuned on higher resolution image, classification pairs that are fewer in number



back propagation of loss to update learnable parameters

## Inference ViT



# The quadratic cost

The attention matrix is  $N \times N$ , so both compute and memory grow with **the square of the number of tokens**. For a sentence of 200 words that is irrelevant. For an image it is the single fact that shapes every architecture in Part 5.

| Input       | Patch size        | Tokens N | Entries in A | Relative cost |
|-------------|-------------------|----------|--------------|---------------|
| 224 x 224   | 32 x 32           | 49       | 2,401        | 1x            |
| 224 x 224   | 16 x 16           | 196      | 38,416       | 16x           |
| 384 x 384   | 16 x 16           | 576      | 331,776      | 138x          |
| 512 x 512   | 16 x 16           | 1,024    | 1,048,576    | 437x          |
| 1024 x 1024 | 16 x 16           | 4,096    | 16,777,216   | 6,988x        |
| 224 x 224   | 1 x 1 (per pixel) | 50,176   | 2.5 billion  | impossible    |

## Why patches exist

The last row is the reason a Vision Transformer does not treat pixels as tokens. Attention over every pixel of even a small image is out of reach on any hardware.

Splitting the image into 16 x 16 patches reduces the token count by a factor of 256, which is exactly what makes the whole idea feasible.

Everything in Part 5 — windowed attention, pooled keys, linear attention — is an attempt to reduce this term without giving up the ability to relate distant regions. Keep the quadratic in mind and those architectures stop looking arbitrary.

# ViT model sizes

| Model        | Layers L | Width d | Heads | MLP size | Params | Roughly comparable to        |
|--------------|----------|---------|-------|----------|--------|------------------------------|
| ViT-Tiny/16  | 12       | 192     | 3     | 768      | 5.7M   | MobileNetV2                  |
| ViT-Small/16 | 12       | 384     | 6     | 1536     | 22M    | ResNet-50                    |
| ViT-Base/16  | 12       | 768     | 12    | 3072     | 86M    | ResNet-152 (but slower)      |
| ViT-Large/16 | 24       | 1024    | 16    | 4096     | 307M   | nothing in the ResNet family |
| ViT-Huge/14  | 32       | 1280    | 16    | 5120     | 632M   | —                            |

## Reading the naming

The number after the slash is the patch size, not a layer count. ViT-B/16 means Base with 16 x 16 patches; ViT-B/32 is the same network reading 32 x 32 patches.

Smaller patches mean more tokens, which means more compute and better accuracy. ViT-B/16 has four times the tokens of ViT-B/32 and roughly four times the attention cost, with the identical parameter count.

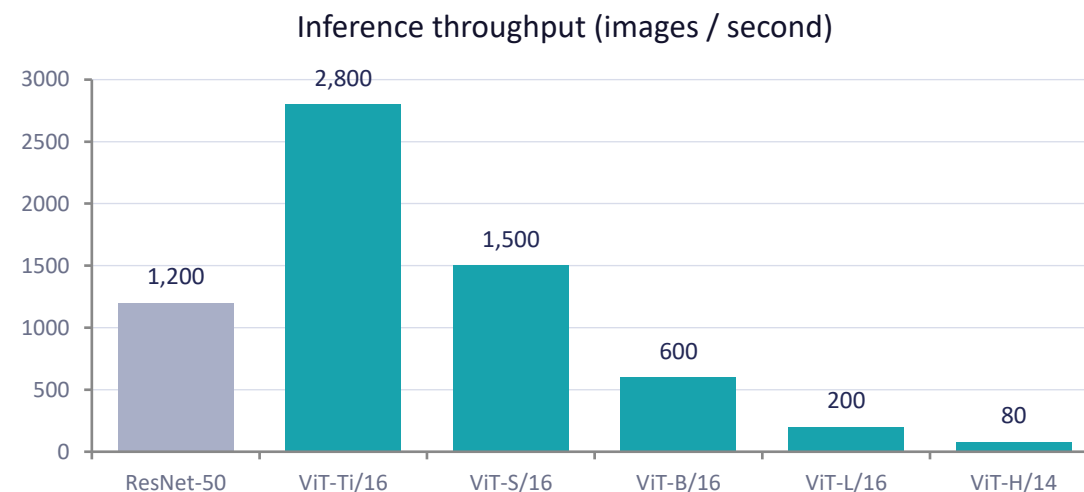
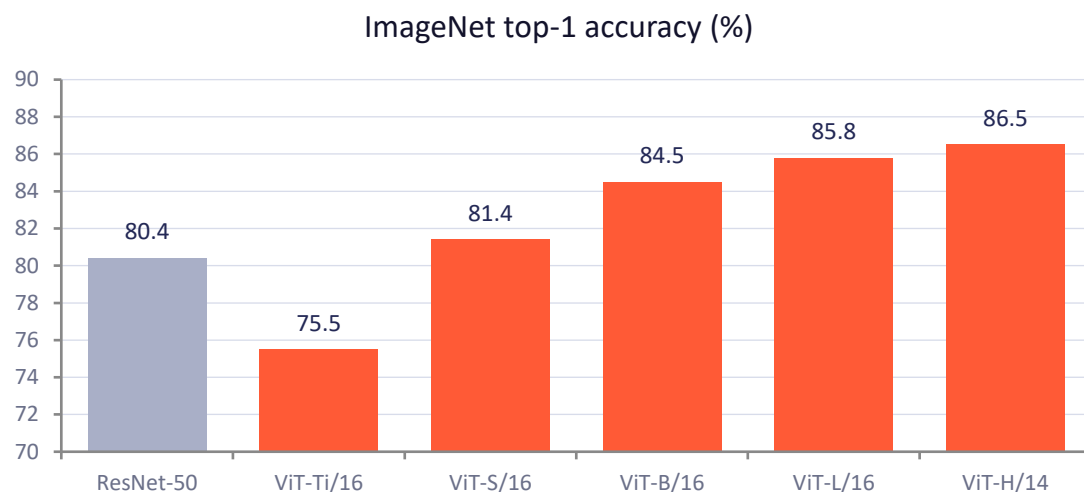
## A note on "comparable"

Parameter count is a poor guide to either speed or accuracy across architecture families. A ViT-Base and a ResNet-152 have wildly different memory access patterns and very different behaviour at different batch sizes.

If throughput matters to your project, measure images per second on your own hardware. Published FLOP counts routinely mislead by a factor of two or more in either direction.

# Choosing a size: what you actually trade

The same architecture at five widths, plus a ResNet-50 for scale. Accuracy is ImageNet-1k top-1 after ImageNet-21k pretraining; throughput is images per second at inference on one modern GPU.



*Figures are approximate and collected from several sources; throughput in particular depends heavily on hardware, precision and batch size. Use them as ratios, not as measurements.*

## ViT-S/16 is the sweet spot

Beats ResNet-50 by a point of accuracy at a comparable speed and a similar parameter count. If you are replacing a ResNet-50 baseline, this is the honest comparison and usually the right default.

## ViT-B/16 if accuracy matters more than latency

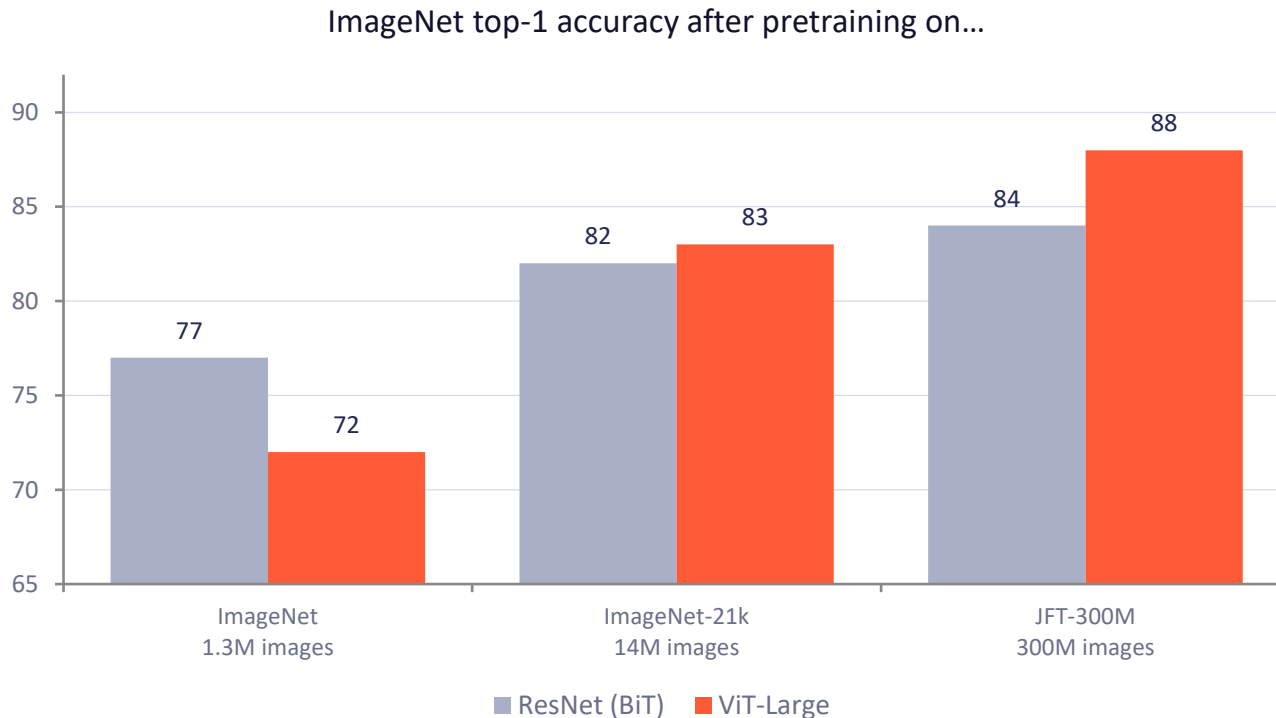
Three points above ViT-S, at roughly a third of the throughput. The standard research default, and the size most pretrained checkpoints are released in.

## L and H only with serious data and compute

The last two points cost an order of magnitude in both. On a departmental GPU and a few thousand images they will underperform ViT-S, not beat it.

# ViTs are data-hungry — and that is the whole story

The original paper trained the same ViT on three datasets of increasing size and compared against a ResNet baseline. The pattern it found is the most important empirical fact in this lecture.



## How to read this

- **At 1M images the ViT loses.** Badly. Without convolutional priors it has to learn locality and translation structure from scratch, and a million images is not enough.
- **At 14M they are level.** The crossover point is somewhere in the tens of millions.
- **At 300M the ViT wins clearly.** And keeps improving with scale where the ResNet is flattening — the priors have become a ceiling rather than a help.

*Figures are approximate and reconstructed from the paper for illustration.*

**The practical corollary for your projects:** you will essentially never train a ViT from scratch. You will download one that someone else pretrained on hundreds of millions of images and fine-tune it — which is exactly why the self-supervised methods in Part 6 matter so much.

# What a trained ViT actually learns

A useful diagnostic is **mean attention distance**: for each head, average the spatial distance between a query patch and the patches it attends to, weighted by the attention scores. Plotting this by layer reveals what the network chose to do with its freedom.

## 1 Early layers do both

In layer 1 some heads have very small attention distance — they behave like a local convolution — while others already span the whole image. The network reinvents locality without being told to, and keeps global capacity alongside it.

## 2 Later layers go global

By the deeper blocks nearly every head attends broadly. Attention distance grows with depth, much as receptive field does in a CNN — except the model was free to do otherwise.

## 3 More data means more locality

Models pretrained on larger datasets develop more local heads in the early layers. The convolutional prior is not wrong; it is simply something a ViT can learn if given enough examples.

## 4 Representations differ from CNNs

Comparing layer representations shows ViTs propagate features far more uniformly across depth, with strong similarity between early and late layers — largely because of the residual path and the constant token resolution.

*The honest summary: given enough data a transformer rediscovers much of what we hand-coded into CNNs, and then goes further. Given little data, it rediscovers nothing and underperforms.*

# ViT versus CNN, honestly

| Property                            | CNN                                                   | Vision Transformer                                            |
|-------------------------------------|-------------------------------------------------------|---------------------------------------------------------------|
| Built-in priors                     | Locality, translation equivariance, hierarchy         | Almost none — only the patch grid                             |
| Receptive field                     | Grows with depth; global only at the end              | Global from the first block                                   |
| Weight sharing                      | One kernel applied at every position                  | Shared projections, but content-dependent mixing              |
| Data efficiency                     | Strong — works with thousands of images               | Weak — needs millions, or pretrained weights                  |
| Cost with resolution                | Linear in pixels                                      | Quadratic in tokens unless the design fixes it                |
| Multi-scale features                | Natural — every stage is a different scale            | Absent in plain ViT; requires a hierarchical variant          |
| Robustness to occlusion / shuffling | Degrades sharply                                      | Notably more robust in several studies                        |
| Texture vs shape bias               | Leans towards texture                                 | Leans more towards shape, closer to human judgement           |
| Interpretability                    | Grad-CAM and filter visualisation                     | Attention maps, often more directly readable                  |
| Best when                           | Data is limited, compute is tight, resolution is high | Large pretrained checkpoints are available; scale is possible |

Neither column is a verdict. The right reading is that these are two different points on a priors-versus-flexibility trade-off, and your dataset size decides which point you want.

# Seven things to take away

1

**Attention is a content-dependent weighted average.**

Queries match keys, softmax turns the matches into weights, values are blended. Everything else is packaging.

2

**Q, K and V are three projections of the same tokens.**

Which is why it is called self-attention, and why the same block works for text, images and audio.

3

**The  $\sqrt{d_k}$  factor keeps softmax from saturating.**

One example of the wider principle: keep the scale of intermediate quantities near one.

4

**Attention is global but position-blind and quadratic.**

Hence positional encodings, and hence every efficient-attention design in Part 5.

5

**A ViT is patches plus a language transformer.**

The patch embedding is a stride-16 convolution; after that, nothing is vision-specific.

6

**Priors trade against data.**

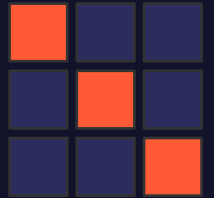
Below roughly ten million images the CNN priors win; above it they become a ceiling. Your dataset size decides.

7

**Much of the gain was the training recipe.**

ConvNeXt showed a plain ResNet with transformer-era training matches Swin. Be sceptical of architecture-only claims.

# References and further reading



## Foundational

- Vaswani et al. — Attention Is All You Need (NeurIPS 2017)
- Dosovitskiy et al. — An Image is Worth 16x16 Words (ICLR 2021)
- Touvron et al. — DeiT: Data-efficient Image Transformers (ICML 2021)
- Liu et al. — Swin Transformer (ICCV 2021)
- Xie et al. — SegFormer (NeurIPS 2021)
- Liu et al. — A ConvNet for the 2020s (CVPR 2022)

## Pretraining and analysis

- He et al. — Masked Autoencoders Are Scalable Vision Learners (CVPR 2022)
- Caron et al. — Emerging Properties in Self-Supervised ViTs / DINO (ICCV 2021)
- Oquab et al. — DINOv2 (2023)
- Radford et al. — Learning Transferable Visual Models / CLIP (ICML 2021)
- Raghu et al. — Do Vision Transformers See Like CNNs? (NeurIPS 2021)
- Xiao et al. — Early Convolutions Help Transformers See Better (NeurIPS 2021)

## Practical

- timm (pytorch-image-models) — pretrained ViT, Swin and hybrid checkpoints
- The Annotated Transformer (Harvard NLP) — line-by-line implementation
- Karpathy — nanoGPT, for a minimal readable attention implementation
- Lucas Beyer — Vision Transformer lecture slides and notes
- HuggingFace Transformers — ViT, SegFormer and Swin model cards